

UNIVERZA V LJUBLJANI
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Nik Katanović Leban

**Prototip spletne aplikacije za prodajo
vozil z ocenjevanjem njihove vrednosti**

DIPLOMSKO DELO

VISOKOŠOLSKI STROKOVNI ŠTUDIJSKI PROGRAM
PRVE STOPNJE

MENTOR: viš. pred. dr. Igor Rožanc

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani <http://www.gnu.org/licenses/>.

Besedilo je oblikovano z urejevalnikom besedil $\text{\LaTeX}$.

Kandidat: Nik Katanović Leban

Naslov: Prototip spletne aplikacije za prodajo vozil z ocenjevanjem njihove vrednosti

Vrsta naloge: Diplomaska naloga na visokošolskem programu prve stopnje Računalništvo in informatika

Mentor: viš. pred. dr. Igor Rožanc

Opis:

V diplomskem delu zasnujete prototip spletne aplikacije za prodajo vozil z vključeno samodejno oceno tržne vrednosti vozil na podlagi primerljivih oglasov na spletu. V ta namen preučite obstoječe rešitve, opredelite zahteve in zasnujete aplikacijo, pri čemer posebno pozornost namenite učinkovitemu vrednotenju vozil. Z uporabo ustreznih tehnologij implementirajte delujoč prototip ter ga ustrezno analizirajte in ovrednotite.

Title: Prototype of a web application for selling vehicles with vehicle valuation

Description:

In the thesis, design a prototype of a web application for vehicle sales with integrated automated market value estimation based on comparable online listings. For this purpose, examine existing solutions, specify the requirements, and design the application, with a special focus on effective vehicle valuation. Implement a working prototype using appropriate technologies, then analyze and evaluate it accordingly.

Rad bi se zahvalil svojemu mentorju Igorju Rožancu, za vso aktivno pomoč in svetovanje pri razvoju te diplomske naloge. Zahvalil bi se tudi vsem sodelavcem iz podjetja CargoX, ki so z mano delili svoje znanje in izkušnje, ter mi omogočili, da sem se lahko lotil tako zahtevnega projekta. Posebna zahvala gre moji družini, ki me je ves čas podpirala in mi stala ob strani.

Kazalo

Povzetek

Abstract

1	Uvod	1
2	Sorodne rešitve	3
3	Ozadje	7
3.1	Oglas in atributi vozila	7
3.2	Zajem podatkov iz spletnih virov	9
4	Razvoj rešitve	13
4.1	Zahteve	13
4.2	Arhitektura rešitve	15
4.3	Uporabne tehnologije	16
4.4	Načrtovanje podatkovne baze	18
4.5	Arhitektura zalednega sistema	22
4.6	Arhitektura uporabniškega vmesnika	26
4.7	Sistem za vrednotenje vozil	30
4.8	Infrastruktura in avtomatizacija	37
5	Analiza in vrednotenje rešitve	43
5.1	Raziskovalna vprašanja in merila uspešnosti	43
5.2	Metodologija in merilno okolje	44

5.3	Uporabljene mere	45
5.4	Značilnosti zbranih podatkov	46
5.5	Natančnost ocene vrednosti	50
5.6	Občutljivost algoritma na parametre	52
5.7	Latenca posameznega vrednotenja	54
5.8	Obnašanje pod obremenitvijo	55
5.9	Preverjanje nefunkcionalnih zahtev	56
5.10	Preverjanje funkcionalnih zahtev z uporabniškim preizkusom	58
5.11	Simulacija uporabe	59
6	Sklep	63
	Literatura	67
	Članki v revijah	67
	Članki v zbornikih konferenc	69
	Ostala literatura	70
	Dodatek	75

Povzetek

Naslov: Prototip spletne aplikacije za prodajo vozil z ocenjevanjem njihove vrednosti

Avtor: Nik Katanović Leban

Diplomsko delo obravnava problem določanja ustrezne tržne vrednosti rabljenega vozila, pri prodaji ali nakupu vozila, zaradi nezadostnega pregleda stanja na trgu, stranke težko ocenijo primernost ponujene cene in vozilo kupijo po previsoki ceni ali ga prodajo pod njegovo tržno vrednostjo. Zato smo razvili prototip spletne aplikacije za objavo in pregled oglasov vozil, ki uporabnikom omogoča enostavno in varno prodajo ter nakup vozil. Ključen del predlagane rešitve je sistem za avtomatsko ocenjevanje vrednosti vozil, ki temelji na zbiranju podatkov iz zunanjih virov, njihovi normalizaciji in primerjavi vozila s podobnimi vozili na trgu. Pri oceni vrednosti se upoštevajo ključne lastnosti vozila, kot so znamka, model, letnik in prevoženi kilometri. Za pridobivanje podatkov so bili uporabljeni javno dostopni podatkovni viri ter postopki samodejnega zajema podatkov s spletnih strani za prodajo vozil. Zbrane podatke sistem obdela in uporabi za izračun ocenjene tržne vrednosti vozila. Rezultat diplomskega dela je delujoč prototip spletne aplikacije za prodajo vozil s podporo za samodejno vrednotenje vozil, razvit z uporabo tehnologij React [41], TypeScript [43], Node.js [47], Express [10], Python [53], PostgreSQL [65], Redis [57] in Docker [14].

Ključne besede: spletna aplikacija, vozila, vrednotenje, React, Express, Node.js, PostgreSQL, Docker.

Abstract

Title: Prototype of a web application for selling vehicles with vehicle valuation

Author: Nik Katanović Leban

This thesis addresses the problem of determining the appropriate market value of a used vehicle. When selling or buying a vehicle, customers have an insufficient overview of the situation on the market, so they find it difficult to assess whether the offered price is reasonable and end up buying a vehicle at too high a price or selling it below its market value. We therefore developed a prototype of a web application for publishing and browsing vehicle listings, which enables users to sell and buy vehicles simply and safely. A key part of the proposed solution is a system for automatic vehicle valuation, based on collecting data from external sources, normalising it, and comparing a vehicle with similar vehicles on the market. The valuation takes into account key vehicle characteristics such as make, model, year, and mileage. Publicly available data sources and automated data collection from vehicle sales websites were used to obtain the data. The system processes the collected data and uses it to compute the estimated market value of a vehicle. The result of the thesis is a working prototype of a web application for selling vehicles with support for automatic vehicle valuation, developed using the technologies React [41], TypeScript [43], Node.js [47], Express [10], Python [53], PostgreSQL [65], Redis [57], and Docker [14].

Keywords: web application, vehicles, valuation, React, Express, Node.js, PostgreSQL, Docker.

Poglavje 1

Uvod

Nakup in prodaja rabljenih vozil predstavljata pomemben del v trgu rabljenih vozil. Pri tem se kupci in prodajalci pogosto srečujejo s težavo določanja primerne tržne vrednosti vozila, saj nimajo zadostnega pregleda nad trenutno ponudbo in gibanjem cen na trgu. Raznolikost informacij je še posebej izrazita, ker je vsako vozilo unikatno glede na svoje stanje, opremo in zgodovino uporabe [24, 19].

Najpogostejši pristop k oceni vrednosti je zato primerjava primerljivih oglasov podobnih vozil, vendar je samostojno izvajanje takšne primerjave časovno potratno in nezanesljivo. Oglasi lahko vsebujejo nerealno visoke ali nizke cene, uporabniki pa pogosto nimajo dovolj znanja in izkušenj, zato lahko vozilo prodajo pod njegovo tržno vrednostjo ali pa ga kupijo po previsoki ceni. Na trgu že obstajajo rešitve za ocenjevanje vrednosti vozil, vendar so mnoge plačljive, prilagojene tujim trgom ali pa temeljijo na zastarelih podatkih, zaradi česar njihove ocene pogosto ne odražajo dejanskega stanja na slovenskem trgu.

Cilj diplomskega dela je razvoj prototipa spletne aplikacije za objavo in pregled oglasov vozil, ki vsebuje tudi sistem za samodejno ocenjevanje vrednosti vozil na podlagi primerljivih oglasov na trgu [38]. Dolgoročni cilj sistema je privabiti zadostno število uporabnikov, da na platformi objavijo lastne oglase vozil, s čemer bo potreba po podatkih iz zunanjih virov postopno upadla, tr-

žno vrednost vozil pa bo mogoče ocenjevati na podlagi preverjenih oglasov, ki so zbrani znotraj platforme same.

Na začetku diplomskega dela si zastavljamo izhodišče, da je mogoče s pomočjo zbranih tržnih podatkov in primerjalne analize podobnih vozil. Uporabniku ponuditi uporabno in dovolj natančno oceno tržne vrednosti vozila. Pri tem želimo preveriti, ali vključitev takšnega sistema v prototip izboljša uporabniško izkušnjo pri nakupu in prodaji vozil.

Preostanek diplomskega dela je organiziran na naslednji način. Poglavje 2 predstavi podobne rešitve za prodajo in vrednotenje vozil. Poglavje 3 podrobneje predstavi oglaševanje vozil, na katerem temelji razvita rešitev, in postopke zajema podatkov s spletnih virov. Poglavje 4 je jedro naše rešitve in opiše opredelitev zahtev, načrtovanje podatkovne baze in arhitekture sistema, implementacijo zalednega in odjemalskega dela aplikacije, delovanje algoritma za vrednotenje vozil ter infrastrukturo in avtomatizacijo namestitve. V poglavju 5 je napisana analiza rešitve, diplomsko delo pa se zaključi s povzetkom doseženih rezultatov, kritično presojo omejitev rešitve in smeri za nadaljnje izboljšave.

Poglavje 2

Sorodne rešitve

Na področju prodaje rabljenih vozil obstaja več spletnih aplikacij, ki uporabnikom omogočajo objavo in pregled oglasov vozil. Pokaže pa se, da imajo le nekatere možnost vrednotenja vozil, od teh je večina plačljivih. Med najbolj znanimi tujimi ponudniki sta **mobile.de** [44] in **AutoScout24** [2], ki združujeta veliko število oglasov iz različnih držav ter uporabnikom omogočata filtriranje vozil po številnih kriterijih. Poleg njiju obstaja še vrsta drugih platform, ki uporabnikom bodisi zgolj omogočajo prodajo vozil bodisi ponujajo oceno njihove vrednosti, med njimi so **AutoTrader** [1], **La Centrale** [36], **Otomoto** [49], **Cars.com** [8] in **Kelley Blue Book** [35].

Za slovenski trg je najbolj razširjena platforma **AvtoNet** [3], ki je izšla leta 1997 in danes velja za prevladujočo aplikacijo za prodajo rabljenih vozil na slovenskem trgu, ter predstavlja naše osrednje mesto za objavo in iskanje oglasov. Uporabniku omogoča prodajo raznovrstnih vozil (kot so avtomobili, motorna kolesa in plovila). Kljub dolgi prisotnosti na trgu ni doživela večjih posodobitev, zato je njen uporabniški vmesnik zastarel in ponuja precej prostora za izboljšave. Za objavo oglasa se mora uporabnik prijaviti, medtem ko za brskanje po oglasih prijava ni potrebna. Prikaz posameznega oglasa na **AvtoNet-u** [3] je predstavljen na sliki 2.1.

Audi A6 Avant 3.0 TDI QUATTRO-235KW-BI-TURBO-MATRIX-NAVI-S LI..
☆



1.registracija	2015
Prevoženih	258630 km
Gorivo	diesel motor
Menjalnik	avtomatski menjalnik
Motor	2967 cm, 235 kW / 320 KM

AKCIJSKA CENA

18.295 €

16.995 €



Slika 2.1: Primer oglasa na Avto.net-u

Uporabniški vmesnik je pregleden in omogoča enostavno orientacijo, zato se uporabnik na njem hitro znajde. Glavne pomanjkljivosti so predvsem manjkajoče dodatne funkcionalnosti, ki bi izboljšale uporabniško izkušnjo. Dober primer je obrazec za objavo oglasa. Vneseni podatki se recimo ob prehodu na naslednji korak ne shranijo, zato mora uporabnik ob vrnitvi na prejšnji korak, ponovno vpisati celotno vsebino. Obrazec je tudi razmeroma dolg in uporabniku ne prikazuje, koliko korakov ima še do zaključka objave. Platforma sicer dela zanesljivo in zadovoljuje osnovne potrebe uporabnikov. Uporabniški vmesnik za objavo oglasa je prikazan na sliki 2.2.

Avto
Moto
Gospodarska vozila
Mehanizacija

Izberite osnovne podatke o vozilu

Znamka:

Model:

Karoserijska oblika:

Prva registracija:

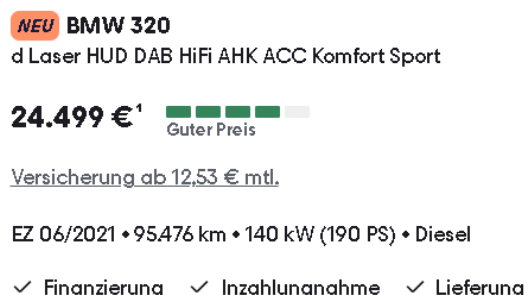
Pogonsko gorivo:

☐ bencin
 ☒ diesel
 ☐ hibridni pogon
 ☐ e-pogon
 ☐ LPG avtoplin
 ☐ CNG zemeljski plin

Klikni za nadaljevanje

Slika 2.2: Objava oglasa na Avto.net-u

Mobile.de [44] ima novejši uporabniški vmesnik, ki je bolj pregleden in omogoča enostavno navigacijo po spletnem mestu. Uporabniku omogoča, da objavi oglas, brska po oglasih in filtrira vozila po različnih kriterijih. Za razliko od **AvtoNet-a** [3], **Mobile.de** [44] omogoča tudi oceno vrednosti vozila. Pri posameznih oglasih prikazuje tudi mnenje, ali je cena ugodna, primerna ali previsoka. Mnenje je predstavljeno z petstopenjsko barvno lestvico: pri ugodni ceni je lestvica obarvana z zeleno, pri previsoki z rdečo, ostale stopnje pa so označene z vmesnimi barvami. Takšen pristop uporabniku omogoča hitrejše odločanje, saj mu ni treba ročno primerjati oglasov med seboj. Ocena temelji na samodejni primerjavi s podobnimi vozili v ponudbi. Pri tem pa platforma ne ponuja možnosti samostojnega vrednotenja vozila na podlagi uporabniško vnesenih podatkov, kadar oglas še ne obstaja. Prikaz osnovnih podatkov vozila z mnenjem je predstavljen na sliki 2.3.

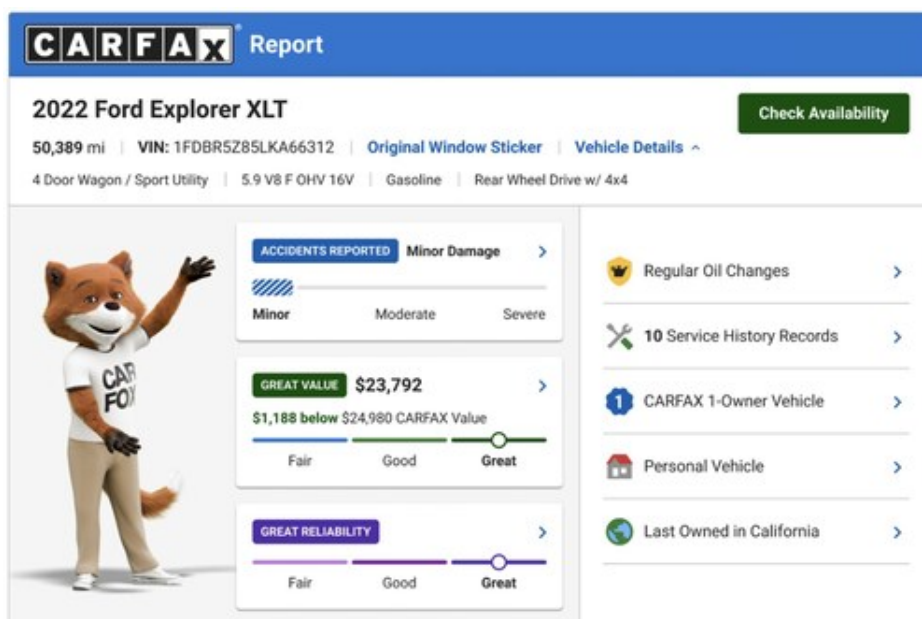


Slika 2.3: Primer ocene vozila na Mobile.de

Poleg spletnih aplikacij za oglaševanje vozil obstajajo tudi specializirane storitve za vrednotenje vozil, med najbolj znanima sta **Eurotax** [19] in **CARFAX** [7]. **Eurotax** [19] se pogosto uporablja v industriji vozil, zavarovalništvu in pri trgovcih z vozili. Njegove ocene temeljijo na obsežnih zbirkah podatkov o vozilih in tržnih cenah, vendar je dostop do storitve praviloma plačljiv. Zaradi poslovnega modela in omejenega dostopa podrobna analiza delovanja sistema ni mogoča.

Na severnoameriškem trgu je najbolj razširjena tovrstna storitev **CARFAX** [7], ki poleg vrednotenja vozila ponuja tudi vpogled v njegovo celotno zgodovino.

Vsako vozilo je določeno z enolično identifikacijsko številko (VIN) [32], na podlagi katere storitev pripravi poročilo o zgodovini vozila. To poročilo vključuje podatke o številu prejšnjih lastnikov, servisni zgodovini, zgodovini registracij, prijavljenih prometnih nesrečah in škodi. **CARFAX** [7] ponuja funkcijo vrednotenja vozila, ki za razliko od preprostih ocen, ki upoštevajo zgolj znamko, model in letnik, vključuje zgodovino vozila, prevožene kilometre, škodo zaradi nesreč, servisne posege ter uporabo vozila (osebna, najemniška ali komercialna uporaba). Vrednost se prilagaja tudi glede na regijo, saj se ponudba in povpraševanje razlikujeta po posameznih območjih. Kljub obsežni funkcionalnosti je storitev žal prilagojena za severnoameriški trg, zato ni neposredno uporabna za nas. V naslednji sliki 2.4 je prikazan oglas od **CARFAX-a** [7], ki vključuje oceno vrednosti vozila in splošno oceno obrabe avtomobila.



Slika 2.4: Primer ocene vozila na CARFAX

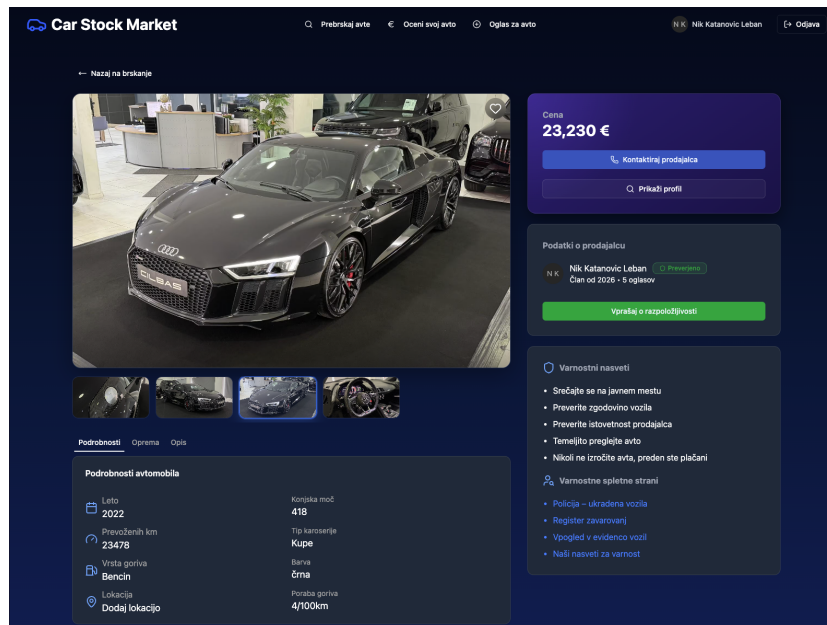
Poglavje 3

Ozadje

V tem poglavju predstavimo temeljne pojme in postopke, na katerih temelji naše delo. Najprej natančneje opišemo ključne tehnologije s tega področja, kaj je oglas rabljenega vozila in kateri atributi vozila so za nas relevantni. V nadaljevanju predstavimo še pristope za zajem podatkov, s katerimi smo pridobili gradivo za sistem vrednotenje vozil.

3.1 Oglas in atributi vozila

Oglas je vsebina, ki je namenjena predstavitvi izdelka ali storitve, ki se trži, prodaja ali kupuje. Oglaševanje ima dolgo zgodovino; tiskani oglasi so se v evropskih časopisih redno pojavljali že od nekdanj, množično pa se je začelo oglaševati z časopisi in revijami v 19. in 20. stoletju. Z razširjenostjo interneta se je oglaševanje večinoma preneslo v elektronsko obliko, kjer danes poteka pretežni del prodaje in nakupa rabljenih vozil. V našem primeru obravnavamo spletne oglase za prodajo vozila. Vsebovati morajo vse ključne podatke: katero vozilo se prodaja in njegove tehnične značilnosti, ki kupcu pomagajo pri odločitvi o nakupu. Poleg tega mora oglas vsebovati še tržno ceno in kontaktne podatke prodajalca. Oglas ostane objavljen na izbranem mestu, do odločitve umika ali nakupa vozila. Na sliki 3.1 prikazujemo primer oglasa v našem prototipu.



Slika 3.1: Primer oglasa na prototipu

3.1.1 Atributi vozila

Atributi vozila, ki vplivajo na ceno, so njegove tehnične ali fizične lastnosti. To pa so znamka, model, izvedba (angl. *trim*), barva, opis in prostornina motorja, vrsta goriva, vrsta menjalnika, pogon (sprednji, zadnji ali štirikolesni), letnik izdelave, datum prve registracije, prevoženi kilometri, moč motorja (izražena v kilovatih ali konjskih močeh), emisijski razred, poraba goriva, tip karoserije, število vrat, število sedežev, število prestav, število lastnikov, servisna zgodovina, stanje vozila (rabljeno, novo, poškodovano), država izvora, oprema (npr. klimatska naprava, navigacija, usnjeni sedeži) in cena. V naši rešitvi se moramo omejiti, zato se osredotočimo predvsem na nekaj ključnih atributov: znamko, model, prevoženi kilometri, letnik izdelave in ceno. **Znamka** in **model** služita kot identifikatorja vozila, saj zaradi niju vemo, za katero vozilo gre. Naslednja dva atributa sta tista, ki najbolj vplivata na vrednost vozila. **Prevoženi kilometri** so pomembni, ker izražajo uporabo vozila; večje kot je število prevoženih kilometrov, večja je verjetnost okvar in

obrade. **Letnik izdelave** pove starost vozila in s tem nakazuje pričakovane prihodnje stroške vzdrževanja. **Cena** je glavni podatek, saj zagotavlja primerjalno vrednost med vozili. Ti atributi so hkrati tudi tisti, ki jih uporabnik pri iskanju vozila najprej pogleda in po katerih filtrira ponudbo. Vseh pet izbranih atributov je razvidnih tudi iz primera oglasa v našem prototipu na sliki 3.1, kjer so prikazani na vidnem mestu ob vozilu.

3.2 Zajem podatkov iz spletnih virov

Spletni vir je poljubna spletna storitev ali stran, ki ponuja podatke, uporabne za obdelavo, na primer oglasni portal, javna podatkovna zbirka ali programski vmesnik. Zajem podatkov iz spletnih virov predstavlja temeljni korak pri našem pristopu za vrednotenje rabljenih vozil na trgu. Starost in kakovost zbranih podatkov neposredno vpliva na zanesljivost algoritma za ovrednotenje cene [38]. V splošnem ločimo tri pristope za pridobivanje podatkov iz spleta: izkoriščanje javno dostopnih podatkovnih zbirk, dostop prek vmesnikov API ter samodejni zajem podatkov s spletnih strani [38].

3.2.1 Javno dostopne podatkovne zbirke

Javno dostopne podatkovne zbirke (angl. *open datasets*) predstavljajo enega najprimernejših virov za zbiranje strukturiranih podatkov, saj so podatki v njih že organizirani, kategorizirani in pripravljeni za neposredno obdelavo [33]. Primeri takšnih zbirk so odprti vladni portali (kot sta Statistični urad Republike Slovenije (SURS) [62] in Eurostat [18]), akademski repozitoriji ter portali prometnih agencij. Slabost tega pristopa je omejena razpoložljivost aktualnih tržnih podatkov. Javne zbirke običajno ne vsebujejo oglasov z aktualnimi prodajnimi cenami vozil, ki so nujno potrebni za ocenjevanje tržne vrednosti [24]. Za tuje trge sicer obstajajo javno dostopne zbirke oglasov, pridobljene z oglasnih portalov (na primer za nemški trg zbirka eBay Kleinanzeigen [48]), za slovenski trg pa takšna javna zbirka ne obstaja.

3.2.2 Dostop prek vmesnikov API

Dostop do podatkov prek vmesnika API (angl. *application programming interface*) običajno poteka tako, da se uporabnik najprej registrira pri ponudniku. Ob registraciji uporabnik prejme nosilni žeton (angl. *bearer token*), ki ga mora za namen avtentikacije priložiti vsakemu zahtevku. Večina vmesnikov API je plačljivih, saj ponudniki ustvarjajo prihodek s prodajo podatkov. Brezplačni vmesniki so zato pogosto pomanjkljivi in vsebujejo zastarele podatke. V našem primeru smo vmesnike API uporabili predvsem za začetno polnitev podatkovne baze z vsemi znamkami in modeli vozil, za kar zadostujejo tudi brezplačni ponudniki.

3.2.3 Samodejni zajem podatkov s spletnih strani

Samodejni zajem podatkov s spletnih strani (angl. *web scraping*) je postopek avtomatiziranega pridobivanja strukturiranih informacij iz spletnih strani z razčlenjevanjem njihove vsebine [38]. Ta pristop se pogosto uporablja pri zbiranju oglasov za prodajo rabljenih vozil. Na voljo je veliko spletnih strani, ki vsebujejo obsežne zbirke takšnih oglasov, iz vsakega oglasa pa je mogoče pridobiti ključne strukturirane attribute vozila (npr. znamko, model, letnik izdelave, prevožene kilometre, vrsto goriva, tip menjalnika in navedeno ceno [24]). Zajem poteka z orodji za avtomatizirano brskanje po spletnih straneh (kot so Playwright [42] ali Selenium [60]), ki simulirajo obisk uporabnika, ter z razčlenjevanjem strukture dokumenta HTML za izluščitev relevantnih podatkovnih polj [38]. Problem zajema podatkov na tak način je ta, da so podatki na spletni strani namenjeni za prikaz in ne za posredovanje podatkov uporabnika. Zato se morajo lastniki spletnih strani strinjati z tem, saj so podatki njihova last.

3.2.4 Razčlenjevanje vsebine HTML

Spletna stran je v osnovi dokument v jeziku HTML (angl. *HyperText Markup Language*), ki je organiziran v drevesno strukturo DOM (angl. *Document*

Object Model) [46]. Vsak element dokumenta (naslov, odstavek in podatkovne celice) je v tej strukturi predstavljen kot vozlišče, ki ima lahko starševska in podrejena vozlišča. Orodja za razčlenjevanje (v jeziku Python na primer knjižnici BeautifulSoup [59] in lxml [6]) to strukturo pretvorijo v obliko, primerno za programsko obdelavo.

Za izluščevanje podatkovnih elementov iz razčlenjene strukture se v praksi najpogosteje uporabljata dve vrsti izbirnikov (angl. *selectors*): izbirniki CSS in izbirniki XPath [12]. Izbirniki CSS so bili prvotno zasnovani za oblikovanje spletnih strani, a jih pri zajemu podatkov preusmerimo v poizvedovanje po strukturi DOM [46]. Izbirniki XPath (angl. *XML Path Language*) pa predstavljajo namenski poizvedovalni jezik za navigacijo po drevesnih strukturah XML in HTML, ki poleg preprostega iskanja po oznakah in razredih omogoča tudi premikanje navzgor po drevesu, iskanje po vsebini besedilnih vozlišč ter ima vgrajene funkcije (kot sta `contains()` in `position()`) [12].

V kontekstu zajemanja podatkov iz oglasnih portalov za vozila to pomeni, da lahko z izbirniki iz razčlenjenega dokumenta izluščimo vse podatke iz oglasa.

Poglavje 4

Razvoj rešitve

Pred začetkom razvoja smo morali najprej jasno opredeliti zahteve. Sledila je zasnova načrta rešitve, nato izbira ustreznih tehnologij na podlagi lastnih izkušenj in predznanja ter pregleda forumov in strokovnih člankov. Ko smo zaključili razvoj prototipa aplikacije, smo se lotili še ločenega razvoja sistema za vrednotenje vozil v programskem jeziku Python.

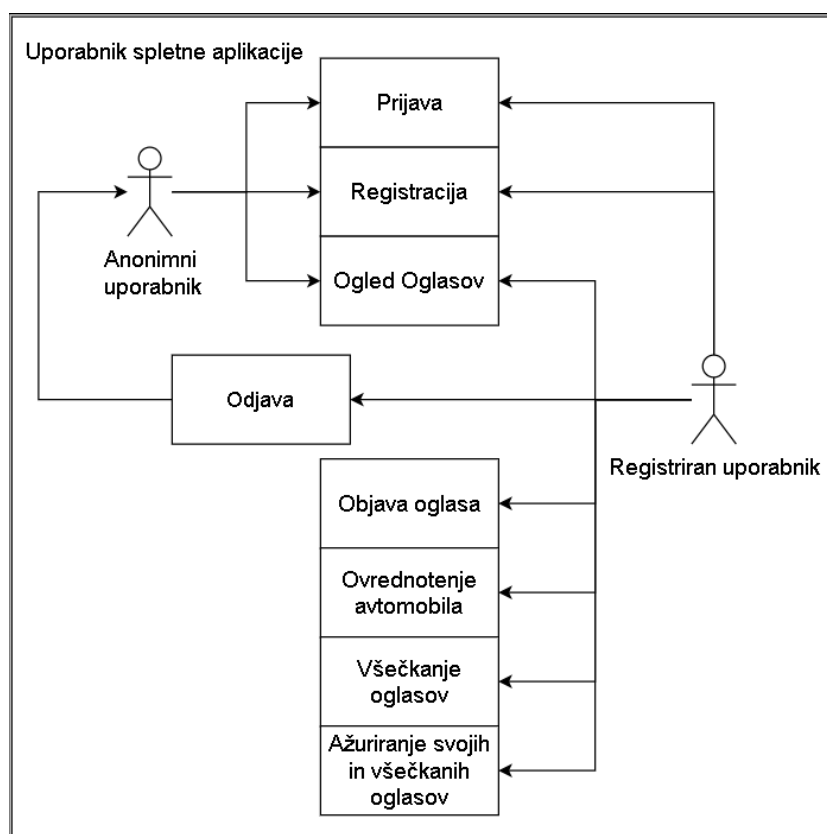
4.1 Zahteve

Pred začetkom razvoja smo opredelili zahteve, ki jih delimo na funkcionalne in nefunkcionalne. Pri opredelitvi funkcionalnih zahtev ločimo dve vrsti uporabnikov: **anonimnega** in **registriranega**. Anonimni uporabnik lahko dostopa le do pregleda oglasov, medtem ko je registriranemu uporabniku omogočen dostop do celotne funkcionalnosti sistema. Funkcionalne zahteve so naslednje:

- Uporabnik se lahko registrira.
- Registriran uporabnik se lahko prijavi in odjavi.
- Oglaševanje vozil je omogočeno le registriranim uporabnikom.
- Registriran uporabnik lahko oceni tržno vrednost vozila na podlagi njegovih ključnih atributov (poglavje 3.1.1).

- Registriran uporabnik lahko všečka oglase drugih uporabnikov.
- Registriran uporabnik lahko ažurira svoje objavljene in všečkane oglase.

Na sliki 4.1 prikazujemo diagram primerov uporabe, ki povzema interakcije registriranega in anonimnega uporabnika s sistemom.



Slika 4.1: Diagram primerov uporabe za registriranega in anonimnega uporabnika.

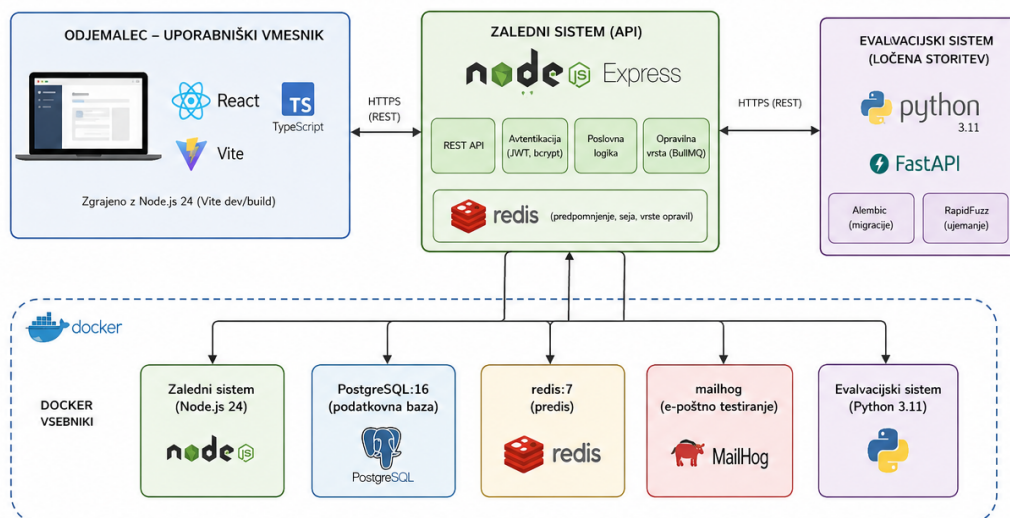
Nefunkcionalne zahteve so naslednje:

- Spletna aplikacija je hitra in odzivna. Uporabniški vmesnik se na dejanja uporabnika odzove praktično takoj (v manj kot 100 ms), zahtevki HTTP proti zalednemu sistemu (z izjemo vrednotenja vozila) pa se v večini primerov obdelajo v manj kot 300 ms.

- Sistem za vrednotenje vozil vrne oceno v kratkem času (manj kot 2 sekundi).
- Uporabniški vmesnik je intuitiven in enostaven za uporabo. Uporabnik lahko glavne naloge (ocena vrednosti vozila, objava oglasa, prijava) opravi brez predhodnih navodil, saj so poti do njih vidne z glavne strani, obrazci pa sproti opozarjajo na napačno vnesene podatke.
- Spletna aplikacija je vizualno privlačna. Uporablja enoten slog (barve, tipografijo in razmike), ki je skladen na vseh pogledih, postavitev pa se prilagaja različnim velikostim zaslonov, od namiznih do mobilnih.
- Sistem ustrezno varuje vse uporabniške podatke (gesla pred shranjevanjem zgosti in uporablja časovno omejene žetone JWT).
- Sistem ostane odziven tudi pri izvajanju zahtevnejših opravil (uporablja asinhrono obdelavo).
- Arhitektura omogoča hitro in vzdržljivo dodajanje novih funkcionalnosti.

4.2 Arhitektura rešitve

Prototip spletne aplikacije je sestavljen iz uporabniškega vmesnika (odjemalec), strežniškega dela aplikacije (zaledni sistem) in ločenega sistema za vrednotenje vozil. Uporabniški vmesnik je razvit z uporabo knjižnice React [41] in programskega jezika TypeScript [43] v okolju Node.js različice 24 [47]. Zaledni sistem je razvit z uporabo ogrodja Express [10] v programskem jeziku JavaScript [20] v okolju Node.js različice 24. Sistem za vrednotenje vozil je implementiran kot ločena storitev v programskem jeziku Python različice 3.11 [53]. Na sliki 4.2 je prikaz arhitekture.



Slika 4.2: Arhitektura prototipa spletne aplikacije

Posamezne komponente tečejo v ločenih vsebnikih na platformi Docker [14] (zaledni sistem, sistem za vrednotenje vozil, PostgreSQL, Redis in MailHog). Razvoj aplikacije je potekal v razvojnem okolju Visual Studio Code. Za avtomatizacijo preverjanja kode, izvajanja testov ter gradnje vsebnikov Docker je bil uporabljen sistem GitHub Actions [25], medtem ko so se periodična opravila izvajala prek cron [64]. Za gostovanje aplikacije je bil uporabljen virtualni strežnik ponudnika Hetzner [28]. Namestitev in upravljanje aplikacije na strežniku je bilo izvedeno s pomočjo platforme Dokploy [16].

4.3 Uporabne tehnologije

Poleg temeljnih tehnologij, ki so relativno znane in jih zato ne predstavljamo posebej, smo med razvojem uporabili še nekaj manjših, a uporabnih knjižnic in orodij za ožje probleme. V nadaljevanju jih na kratko predstavimo.

4.3.1 Upravljanje obrazcev s knjižnico React Hook Form

Uporabniški vmesnik vsebuje več obsežnejših obrazcev za izpolnjevanje. Ročno upravljanje takšnih obrazcev v knjižnici React [41] zahteva, da za vsako polje posebej vzdržujemo stanje, obravnavamo dogodke ob spremembi vrednosti in sami zapišemo logiko preverjanja vnosov, kar hitro privede do obsežne in težko vzdržljive kode.

Zato smo uporabili knjižnico React Hook Form [56]. Knjižnica stanje obrazca upravlja prek nenadzorovanih komponent (angl. *uncontrolled components*) in referenc, kar pomeni, da se ob vsakem pritisku tipke ne izriše celoten obrazec, temveč le tisti deli, ki jih sprememba dejansko zadeva. Posamezno polje prijavimo v obrazec, pri čemer hkrati navedemo še pravila preverjanja vnosa (angl. *validation*), na primer obveznost polja, najmanjšo dolžino ali regularni izraz. Napake so nato na voljo v objektu `errors`, iz katerega jih neposredno izpišemo ob pripadajočem polju. Preverjanje vnosov na strani odjemalca opozori uporabnika na napake, še preden se zahtevek sploh pošlje na strežnik, kar zmanjša število nepotrebnih zahtevkov.

4.3.2 Predpomnjenje strežniškega stanja s knjižnico TanStack Query

Del podatkov, ki jih prikazuje uporabniški vmesnik, izvira iz zalednega sistema in se razmeroma redko spreminja. Značilen primer so sezname znamk in modelov vozil, ki jih potrebujemo v skoraj vsakem obrazcu in filtru. Če bi te podatke ob vsakem prikazu komponente znova pridobivali s strežnika, bi obremenjevali zaledni sistem in podaljševali odzivni čas vmesnika.

Zato smo uporabili knjižnico TanStack Query [63]. Ta odgovore strežnika shrani v predpomnilnik, kjer je vsak odgovor označen s ključem poizvedbe (angl. *query key*). Ko komponenta zahteva podatke z že znanim ključem, jih knjižnica vrne takoj iz predpomnilnika, zahtevke na strežnik pa izvede po potrebi v ozadju, prikaz pa osveži šele, ko prispe nov odgovor. Čas, po katerem se shranjeni podatki obravnavajo kot zastareli, določimo za vsak ključ

posebej.

4.3.3 Testiranje zalednega sistema

Zaledni sistem smo pokrili z avtomatiziranimi testi, ki jih izvajamo z ogrodjem Jest [40]. Jest je testno ogrodje za okolje Node.js [47], ki v enem samem paketu združuje izvajalnik testov, knjižnico trditev (angl. *assertions*) in podporo za simulacijo odvisnosti (angl. *mocking*).

Ker večina logike zalednega sistema ni izpostavljena kot posamezna funkcija, temveč kot končna točka API, smo ogrodju Jest dodali še knjižnico Supertest [37]. Ta omogoča pošiljanje zahtevkov HTTP neposredno na aplikacijo Express, ne da bi strežnik dejansko poslušal na omrežnih vratih. V testu tako opišemo celoten potek zahtevka: ga pošljemo na določeno končno točko API, priložimo žeton JWT ter nato preverimo statusno kodo in vsebino odgovora. Te teste samodejno izvaja tudi cevovod neprekinjene integracije (poglavje 4.8.2).

4.3.4 Približno ujemanje nizov s knjižnico RapidFuzz

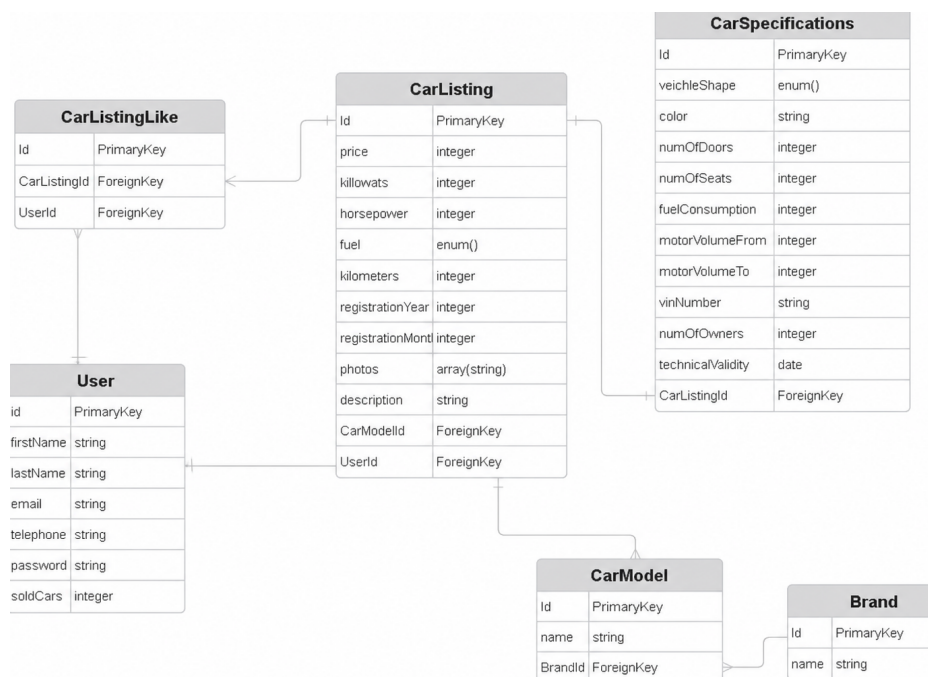
Za približno ujemanje nizov (angl. *fuzzy string matching*) smo uporabili knjižnico RapidFuzz [55], ki podobnost med nizoma izrazi kot oceno med 0 in 100 na podlagi urejevalne razdalje (angl. *edit distance*); zahtevnejše operacije so izvedene v jeziku C++ in zato bistveno hitrejše od enakovrednih izvedb v jeziku Python. Zakaj je približno ujemanje potrebno ter konkreten postopek in izbrane pragove opisujemo v poglavju 4.7.1.

4.4 Načrtovanje podatkovne baze

4.4.1 Načrtovanje podatkovne baze aplikacije

Pri načrtovanju podatkovne baze smo morali natančno premisliti, kako shranjujemo oglase vozil, saj je to ključnega pomena za učinkovito delovanje odločitvenega modela. Previsoka stopnja normalizacije bi lahko upočasnila

poizvedbe, prenizka pa bi vodila do podvajanja podatkov in do težav z zagotavljanjem skladnosti v bazi [17]. Zasnovani podatkovni načrt je predstavljen na sliki 4.3.



Slika 4.3: ER-diagram glavnih entitet aplikacije.

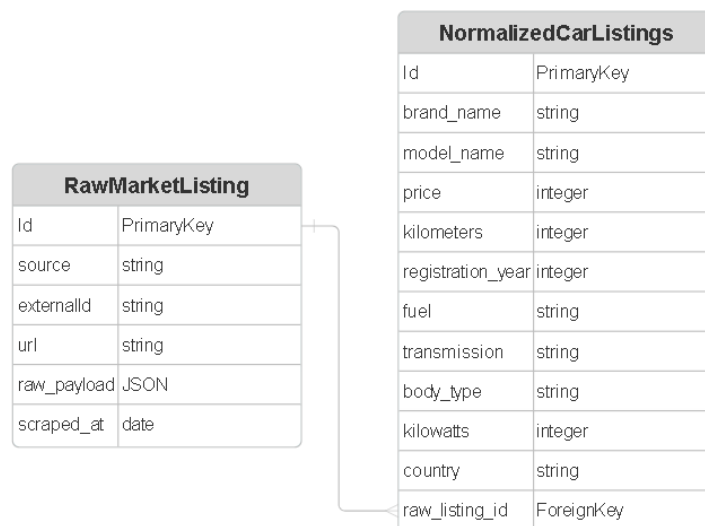
Entiteti **Brand** in **CarModel** sta izločeni v ločeni tabeli, ker med njima obstaja hierarhično razmerje ena-proti-mnogo: ena znamka (**Brand**) ima lahko poljubno število modelov (**CarModel**), vsak model pa pripada natanko eni znamki. Shranjevanje znamk in modelov kot vrednosti tipa `enum` ali prostega niza bi bilo neprimerno iz dveh razlogov. Število možnih vrednosti je namreč preveliko za statično definicijo z `enum`, prosti niz pa bi omogočal podvojene ali nekonsistentne vnose (na primer „Audi A3“ in „audi a3 Sportback“ bi bila obravnavana kot različni entiteti, čeprav gre za isti model). Z relacijsko vezavo prek tujih ključev je zagotovljen referenčni nadzor nad vnesenimi vrednostmi ter enotnost podatkov.

Entiteta **CarSpecifications** je namerno ločena od entitete **CarListing**, ker vsebuje tehnične podrobnosti vozila, ki niso relevantne pri vsakem poizvedovanju. Pri prikazu seznama oglasov (npr. v podatkovni tabeli) zadostujejo le osnovni podatki iz entitete **CarListing** (cena, prevoženi kilometri in letnik), podrobne specifikacije pa se naložijo le ob ogledu posameznega oglasa. Ta ločitev zmanjša količino podatkov, ki se prenesejo iz podatkovne baze pri pogostih poizvedbah, in s tem izboljša odzivnost aplikacije.

Entiteta **CarListingLike** predstavlja asociativno tabelo, ki realizira razmerje mnogo-proti-mnogo med uporabniki in oglasi: en uporabnik lahko vsečka poljubno število oglasov, en oglas pa je lahko vseč pri poljubnem številu uporabnikov. Tabela vsebuje tuji ključ na entiteto **User** in tuji ključ na entiteto **CarListing**, kar zagotavlja referenčno integriteto in preprečuje podvojene vnose. Entiteta **User** hrani ključne podatke o registriranih uporabnikih sistema.

4.4.2 Načrtovanje podatkovne baze za ovrednotenje vozil

V okviru iste podatkovne baze smo dodali ločeno shemo **market**, ki vsebuje dve tabeli, namenjeni zbiranju in obdelavi tržnih podatkov za sistem za vrednotenje vozil (slika 4.4).



Slika 4.4: ER-diagram sistema za vrednotenje vozil.

Tabela **RawMarketListing** služi kot neobdelan arhiv vseh oglasov, ki so pridobljeni iz zunanjih virov – javno dostopnih zbirk podatkov, spletnih vmesnikov API in z zajemanjem spletnih strani. Tabela vsebuje naslednja polja:

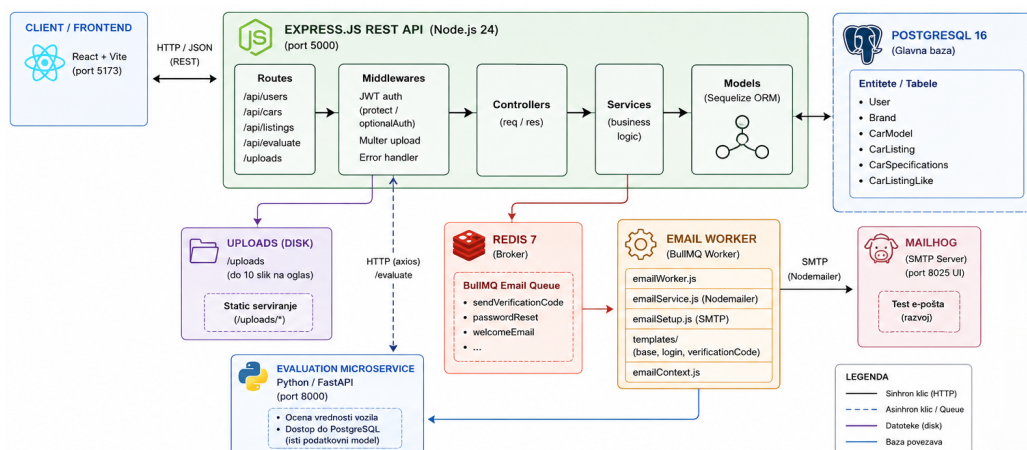
- **source** določa izvor podatkov (npr. ime portala ali vmesnika API),
- **externalId** je identifikator oglasa na izvornem sistemu, ki je določen statično v kodi in preprečuje podvojene vnose,
- **url** hrani neposredno povezavo do oglasa,
- **scraped_at** je časovni žig trenutka zajema podatkov,
- **raw_payload** vsebuje izvorni odgovor v obliki JSON brez kakršnekoli obdelave.

Izvorni odgovor je namerno shranjen v celoti, kar omogoča kasnejšo ponovno obdelavo brez ponovnega zajema podatkov, če se normalizacijska logika spremeni.

Tabela `NormalizedCarListing` vsebuje obdelano in normalizirano obliko oglasov, ki so vezani na tabelo `RawMarketListing` prek tujega ključa `raw_listing_id`. Razmerje med tabelama je ena-proti-mnogo: vsak surovi oglas ima lahko več normaliziranih zapisov, ki jih med seboj loči primarni ključ, vsem pa je skupen tuji ključ na `RawMarketListing`. Polja so poenotena in tipizirana; cena, prevoženi kilometri in letnik so shranjeni kot `integer`, znamka, model, vrsta goriva, tip menjalnika, oblika karoserije in država pa kot `string`. Takšna poenotena struktura omogoča neposredno primerjavo oglasov iz različnih virov pri izračunu tržne vrednosti vozila.

4.5 Arhitektura zalednega sistema

Zaledni sistem smo oblikovali z modularno arhitekturo, ki omogoča jasno ločitev odgovornosti (angl. *separation of concerns*), kar olajša branje, vzdrževanje in nadaljnjo razširljivost kode [13]. Na naslednji sliki je podrobneje prikazana arhitektura zalednega sistema (slika 4.5).



Slika 4.5: Arhitektura zalednega sistema

4.5.1 Modul za poizvedbe v podatkovni bazi

Za implementacijo podatkovnih modelov in poizvedb na podatkovno bazo smo uporabili knjižnico Sequelize [11]. Sequelize je orodje za objektno-relacijsko preslikavo (angl. *Object-Relational Mapping*, ORM), ki preslika objekte iz programske kode v relacijske strukture podatkovne baze [11]. Tak pristop izboljšuje razvoj, saj omogoča programerjem, da namesto neposrednih poizvedb SQL delajo z objekti, kar zmanjša napake in poveča berljivost kode. Prednosti pristopa vključujejo tipsko varnost, avtomatično generiranje poizvedb za različne podatkovne baze ter intuitivnejše upravljanje relacij med entitetami.

Podatkovni modeli so v Sequelize definirani kot razredi, ki predstavljajo tabele v bazi podatkov [11]. Vsak model ima attribute, ki ustrezajo stolpcem v tabeli, ter metode za izvajanje temeljnih operacij s podatki (CRUD – Create, Read, Update, Delete). Ključna pri oblikovanju modelov je tudi postavitve indeksov na poljih, ki se pogosto pojavljajo pri poizvedbah (npr. ID vozila, izvor oglasa ali časovni žigi). Dobro zasnovani indeksi so bistvenega pomena za zagotavljanje hitrih odgovorov poizvedb pri velikih količinah podatkov [23]. Primer postavitve povezave z bazo podatkov je v kodi 1.

Izsek kode 1: Koda za vzpostavitev povezave z bazo podatkov.

```
1 import { Sequelize } from 'sequelize';
2
3 const sequelize = new Sequelize(process.env.DB_NAME, process.env.DB_USER,
4   ↪ process.env.DB_PASSWORD, {
5     host: process.env.DB_HOST,
6     port: process.env.DB_PORT,
7     dialect: 'postgres',
8     logging: false,
9   });
10 export default sequelize;
```

4.5.2 Avtentikacija in avtorizacija uporabnikov

Za avtentikacijo uporabnikov smo implementirali sistem, ki temelji na žetonih JWT (JSON Web Token), kar omogoča varen in enostaven prenos podatkov o uporabnikovi pristnosti [34]. Žeton JWT je strukturiran dokument, ki je sestavljen iz treh delov: glave (angl. *header*), ki vsebuje metapodatke o vrsti kodiranja; *podatkov* (angl. *payload*), ki nosijo informacije o uporabniku; in *digitalnega podpisa*, ki zagotavlja celovitost in pristnost žetona [34]. Postopek avtentikacije je razdeljen na dve fazi. V fazi registracije uporabnik vnese svoje geslo, ki ga pred shranitvijo v podatkovno bazo zgostimo z algoritmom bcrypt [52]. Bcrypt je specializirana kriptografska funkcija, namenjena zgoščevanju gesel. V fazi prijave uporabnik pošlje svoje geslo in naslov elektronske pošte. Zaledni sistem pridobi shranjeno zgoščeno geslo iz baze in ga primerja z novim zgoščenim geslom. Če se gesli ujemata, zaledni sistem generira žeton JWT z informacijami o uporabniku in ga vrne uporabniškemu vmesniku, kjer ga odjemalec shrani v brskalnik, najpogosteje v `localStorage`.

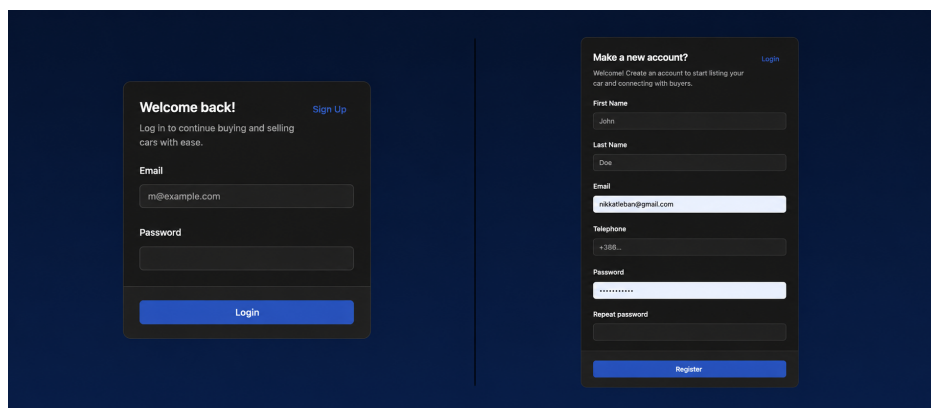
Za zaščito končnih točk API smo implementirali vmesno programsko opremo (angl. *middleware*) za preverjanje žetonov. Gre za funkcijo, ki se izvede pred vsakim zahtevkom in preveri, ali je uporabnik v zahtevku poslal veljaven žeton JWT. Dodatna varnostna plast so časovne omejitve žetona, ki zagotavljajo, da je žeton veljaven samo določen čas. Po preteku roka veljavnosti se mora uporabnik ponovno avtentificirati in pridobiti nov žeton, kar zmanjšuje tveganje ob morebitnem razkritju starejših žetonov [61]. Prijava uporabnika je prikazana v kodi 2, prijava in registracija pa sta na sliki 4.6.

Izsek kode 2: Koda za prijavo uporabnika.

```
1 export const loginUserService = async ({ email, password }) => {  
2   try {  
3     if (!email || !password) throw new MissingUserDataError();  
4  
5     const user = await User.findOne({ where: { email } });
```



```
6
7   if (!user) throw new AppError('Invalid email or password', 401);
8
9   const isPasswordValid = await bcrypt.compare(password, user.password);
10
11  if (!isPasswordValid) throw new AppError('Invalid email or password',
12    ↪ 401);
13
14  const token = jwt.sign({ id: user.id, email: user.email },
15    ↪ process.env.JWT_SECRET, {
16    expiresIn: '7d',
17  });
18
19  await emailQueue.add('login', { type: 'login', to: user.email,
20    ↪ userName: user.firstName });
21
22  return { user, token };
23 } catch (error) {
24   throw error;
25 }
```



Slika 4.6: Prijava in registracija na prototipu.

4.5.3 Asinhrona obdelava podatkov

Za naloge, ki zahtevajo obsežnejšo obdelavo podatkov ali imajo daljši čas izvajanja, smo implementirali asinhrono obdelavo z uporabo sistema čakalnih vrst nalog. Ta pristop omogoča, da strežnik takoj odgovori na zahtevek,

medtem ko se zahtevnejše operacije izvajajo v ozadju kot ločeni procesi. Na ta način se izboljša splošna odzivnost aplikacije. Značilne naloge, ki zahtevajo asinhrono obdelavo, so večinoma zajem podatkov iz zunanjih virov, normalizacija in transformacija zajetih podatkov ter pošiljanje obvestil po elektronski pošti uporabnikom.

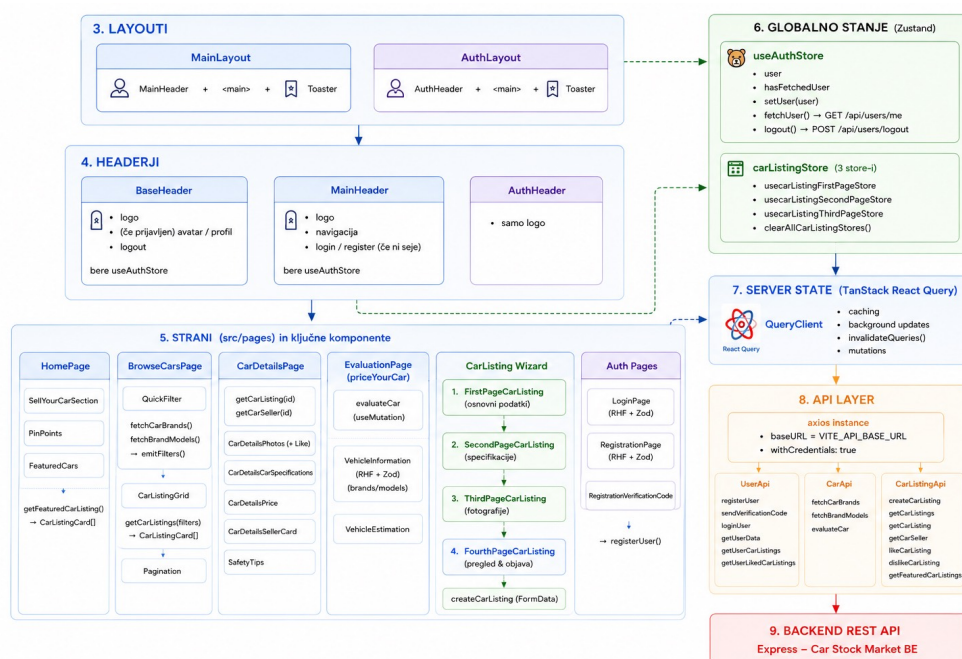
Za upravljanje čakalnih vrst nalog v okolju Node.js smo izbrali knjižnico BullMQ [9], ki zagotavlja zanesljivo upravljanje tovrstnih nalog z vgrajeno podporo za ponovne poskuse (angl. *retry*), časovne omejitve (angl. *timeout*) in obravnavo napak [9]. Sistem temelji na kombinaciji podatkovnega skladišča Redis [57] in delovnih procesov. Redis v tej arhitekturi služi kot posrednik sporočil (angl. *message broker*), ki hrani stanje nalog in omogoča koordinacijo med posameznimi delovnimi procesi [57].

Konfiguracija sistema zahteva tri ključne korake. Najprej se vzpostavi povezava s strežnikom Redis [57]. Nato se registrirajo delovni procesi, ki poslušajo dodeljene vrste nalog, jih obdelujejo in ustrezno posodablajo stanje v skladišču Redis. S takšno zasnovo je zagotovljeno, da se dolgotrajne operacije ne izvajajo na glavni niti aplikacije, kar omogoča sočasno obdelavo in izboljšano razširljivost sistema.

4.6 Arhitektura uporabniškega vmesnika

Uporabniški vmesnik smo zasnovali iz več komponent, ki skupaj sestavljajo celoto. Kot pri zalednem sistemu (poglavje 4.5) smo tudi tu sledili načelu modularnosti, ki omogoča ponovno uporabo komponent in lažje vzdrževanje [50]. Vsako komponento smo oblikovali tako, da opravlja natančno določeno nalogo in prikazuje omejen del vmesnika. Komponente smo namestoma ohranili majhne, saj majhen obseg naravno usmerja razvijalca k temu, da posamezna komponenta ostane osredotočena na eno samo funkcionalnost. Komponente si med seboj izmenjujejo podatke po hierarhiji nadrejenih in podrejenih komponent. Nadrejena komponenta posreduje podatke podrejenim komponentam prek lastnosti (angl. *props*), podrejena komponenta pa nadre-

jeni sporoča spremembe prek povratnih funkcij, ki jih nadrejena komponenta posreduje navzdol. Kot ogrodje za izgradnjo uporabniškega vmesnika smo izbrali React [41]. V naslednjih podglavljih podrobneje predstavimo posamezne dele te arhitekture. Na sliki 4.7 sta prikazani arhitektura in postavitve odjemalskega sistema.



Slika 4.7: Arhitektura odjemalskega sistema

4.6.1 Odjemalska shramba

Določene podatke si mora odjemalec zapomniti, kar pomeni, da jih mora shraniti na svoji strani. Prednost odjemalske shrambe je, da odjemalcu ni treba nenehno pošiljati novih zahtevkov na strežnik, kar občutno zmanjša njegovo obremenitev. Značilen primer je žeton JWT, ki ga odjemalec prejme ob prijavi in ga do odjave oziroma izteka veljavnosti prilaga vsakemu zahtevku (poglavje 4.5.2).

Odjemalska shramba je uporabna tudi pri večkoraknih postopkih, saj si lahko

zapomnimo, do katerega koraka je uporabnik prišel. Če uporabnik namerno ali nenamerno zapre stran, ostane njegovo napredovanje ohranjeno in lahko z delom nadaljuje. Pri shranjevanju podatkov na odjemalcu moramo biti pozorni, da hranimo le neobčutljive podatke ter da stanje pravilno posodabljammo ob vsaki spremembi; sicer se lahko stanje na strani odjemalca razlikuje od stanja na strežniku, kar lahko vodi do napak. Spletni brskalniki za trajno hrambo podatkov na strani odjemalca ponujajo več mehanizmov, med drugim `localStorage` in `sessionStorage`.

Za odjemalsko shrambo smo uporabili knjižnico Zustand [51], ki je preprosta, kompaktna in dobro vzdrževana. Zustand [51] temelji na minimalističnem pristopu k upravljanju stanja in za osnovno delovanje ne zahteva ovojnih komponent (angl. *provider*), kar ga loči od nekaterih drugih uveljavljenih rešitev, kot je recimo Redux [58]. Zaradi majhne velikosti in neodvisnosti od ogrodja React je knjižnica primerna tudi za projekte, pri katerih želimo ohraniti nizko zahtevnost in dobro zmogljivost.

4.6.2 Zahtevki API

Za pošiljanje zahtevkov smo uporabili knjižnico Axios, ki je ena izmed najbolj razširjenih knjižnic za pošiljanje zahtevkov HTTP v okolju JavaScript [20, 4]. Nad njo smo izdelali ovojnico `api`, ki iz okoljskih spremenljivk (angl. *environment variables*) prebere naslov strežnika in ga uporabi kot osnovni naslov URL za vse zahteve. Tak pristop nam omogoča, da osnovni naslov in skupne nastavitve določimo na enem mestu, s čimer se izognemo ponavljanju in poenostavimo morebitne kasnejše spremembe. V kodi 3 je prikazana ovojnica, ki jo nato uvozimo v vsako datoteko, kjer definiramo funkcije za pošiljanje zahtevkov.

Izsek kode 3: Ovojnica Axios za ustvarjanje odjemalca API.

```
1 import axios from "axios";
```

```
2
3 const api = axios.create({
4   baseURL: import.meta.env.VITE_API_BASE_URL,
5   withCredentials: true,
6 });
7
8 export default api;
```

Ovojnico smo nato uporabili pri vsaki definiciji funkcije za pošiljanje zahtevka. Funkcija vrne polje `res.data`, ki vsebuje dejanski odgovor strežnika. Axios namreč vrne objekt s številnimi dodatnimi podatki, ki niso vedno potrebni, zato zaradi preglednosti kode vrnemo le vsebino odgovora [4]. Po potrebi lahko dostopamo tudi do statusne kode strežnika, glave odgovora in konfiguracije zahtevka. Primer definicije funkcije za pridobivanje znamk vozil je prikazan v kodi 4.

Izsek kode 4: Primer definicije funkcije za pridobivanje znamk vozil.

```
1 export const fetchCarBrands = async () => {
2   const res = await api.get<{ brandNames: string[] }>("/api/cars/brands");
3   return res.data;
4 };
```

4.6.3 Razvojno orodje Vite

Vsako izvorno kodo je treba pred prikazom v spletnem brskalniku ustrezno pripraviti oziroma zgraditi. To nalogo v našem projektu opravlja orodje Vite [66], ki omogoča hitro postavitve razvojnega okolja in izgradnjo končne različice aplikacije. Za razliko od starejših orodij za združevanje kode (angl. *bundler*), kot je Webpack [68], ki mora ob vsaki spremembi znova zgraditi večji del kode, Vite tega ne počne in je zato bistveno hitrejši.

Delovanje orodja Vite [66] temelji na dveh ključnih načelih. Prvo je izraba native podpore modulom ECMAScript (angl. *ES modules*) v sodobnih brskalnikih. Ker sodobni brskalniki neposredno razumejo uvožene module, Vite

med razvojem kode ne združuje, temveč posamezne module dostavi brskalniku po potrebi [66]. Drugo načelo je uporaba orodja esbuild [67] za predhodno izgradnjo odvisnosti JavaScript [20]. Orodje je napisano v jeziku Go in je od 10- do 100-krat hitrejše od orodij za združevanje, ki so napisana v jeziku JavaScript.

Pomembna prednost orodja Vite je zmožnost sprotnega osveževanja modulov (angl. *Hot Module Replacement*), pri katerem se osveži le tisti del kode, ki se je spremenil, brez izgube trenutnega stanja in podatkov aplikacije. Ta zmožnost je pri razvoju izjemno pomembna, saj občutno pospeši povratno zanko med spremembo in prikazom ter tako poveča produktivnost.

4.7 Sistem za vrednotenje vozil

Sistem za vrednotenje vozil smo zasnovali kot samostojno storitev v programskem jeziku Python, ki smo ga izbrali zaradi bogatega ekosistema knjižnic za obdelavo in analizo podatkov. Pri razvoju smo osnovnemu jeziku postopoma dodajali pakete glede na potrebe. Za komunikacijo z zalednim sistemom smo potrebovali spletni strežnik, saj je zaledni sistem edini, ki ima dostop do sistema za vrednotenje vozil. Za spletni strežnik in vmesnik API smo uporabili ogrodje FastAPI, ki omogoča enostavno izdelavo končnih točk API v jeziku Python [54]. Na izseku kode 5 je prikazan primer definicije končne točke API, ki vrne seznam znamk vozil iz podatkovne baze.

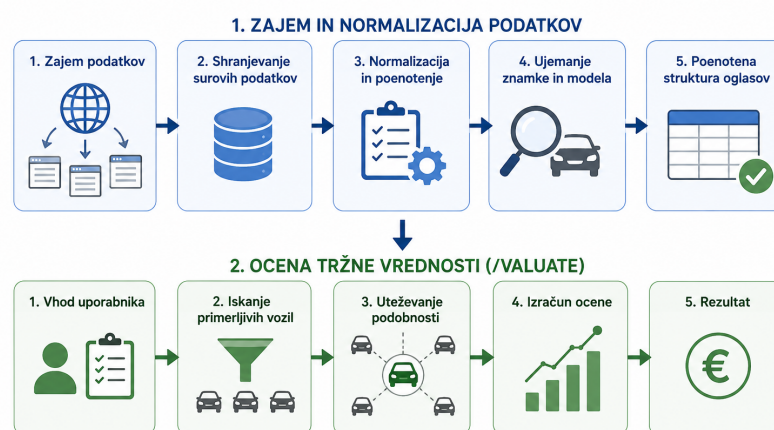
Izsek kode 5: Primer definicije končne točke API z ogrodjem FastAPI

```
1 @app.get("/car-brands")
2 def get_car_brands():
3     return getCarBrands()
```

Podatkovno bazo smo ločili v samostojno shemo z lastnimi modeli, s čimer smo se izognili morebitni zmedi pri poimenovanju in strukturi. Za migracijo tabel in podatkov smo dodali paket Alembic, ki je zmogljivo orodje za

upravljanje in posodabljanje podatkovnih baz [5]. Alembic sledi shemi podatkovne baze in samodejno ustvarja migracijske skripte, ki jih je mogoče verzionirati skupaj z izvirno kodo, kar zagotavlja ponovljivo in nadzorovano spreminjanje strukture podatkovne baze. Za periodični zagon programov za samodejni zajem podatkov (dnevno oziroma tedensko) smo uporabili tehnologijo cron, ki izvaja ukaze ob vnaprej določenih časovnih intervalih.

4.7.1 Algoritem za primerjavo vozil



Slika 4.8: Algoritem za primerjavo vozil

Opis algoritma v nadaljevanju sledi korakom s slike 4.8, od leve proti desni in od zgornje vrstice k spodnji. Začeli smo z zajemom podatkov o oglasih vozil, bodisi s samodejnim zajemom s spletnih strani za prodajo vozil bodisi z dostopom do javno dostopnih virov. Sam postopek zajema smo podrobneje opisali v poglavju 3.2; poganja ga cron. Tu se osredotočamo na nadaljnjo obdelavo zajetih podatkov.

Zajete podatke najprej shranimo v surovi obliki, tako kot pridejo z vira, s čimer zajem ločimo od obdelave (razloge smo pojasnili pri načrtovanju podatkovne baze). Sledi normalizacija, s katero surove podatke pretvorimo v enotno obliko. Za vsak spletni vir smo morali napisati ločen postopek nor-

malizacije. Za vsako vozilo normaliziramo in shranimo ceno, prevožene kilometre, znamko, model, moč motorja, vrsto menjalnika in vrsto goriva. Pri normalizaciji smo vse te podatke sproti izpisovali in poskrbeli za zanesljiv sistem obvladovanja napak. Zlasti v začetni fazi razvoja namreč pogosto prihaja do napak, za njihovo učinkovito odpravljanje pa je potreben podroben izpis poteka izvajanja.

Poleg razlik v strukturi se posamezni viri med seboj razlikujejo tudi v zapisu posameznih vrednosti. Cene so lahko zapisane z ločilom tisočic, ki jih je treba pred pretvorbo v število odstraniti. Prevoženi kilometri so pri enem viru zapisani kot celo število, pri drugem pa kot niz z enoto (na primer „123.456 km“). Datum prve registracije je pri enem viru zapisan v obliki *mesec/leto*, pri drugem pa *leto/mesec*. Podobno je bilo treba poenotiti kategorije za tip menjalnika in vrsto goriva, saj so viri te podatke podajali v različnih jezikih. Poseben korak v tej fazi je ujemanje znamke in modela vozila, ki je hkrati največji izziv normalizacije, saj sta ista znamka ali model na različnih virih pogosto zapisana drugače (okrajšave, dodatne besede, jezikovne razlike – na primer nemško „3er“ ali „Klasse“/„class“).

Zato smo zgradili sistem ujemanja, ki surovo ime najprej poskuša natančno ujeti s kanoničnim imenom v podatkovni bazi, nato preveri tabelo že znanih psevdonimov, šele nazadnje pa uporabi približno (angl. *fuzzy*) ujemanje nizov [45] s knjižnico **RapidFuzz** (poglavje 4.3.4). Prag ujemanja smo nastavili na 90 % za znamke in 75 % za modele. Nižji prag za modele smo izbrali zato, ker so imena modelov krajša in bolj podvržena lažnim ujemanjem. Vsako uspešno približno ujemanje se samodejno shrani kot nov psevdonim za dani vir, s čimer se sistem sproti uči. Tako se vsak naslednji pojav istega zapisa razreši že v fazi iskanja psevdonimov, s čimer se izognemo počasnejšemu približnemu ujemanju. Del kode, ki izvaja ujemanje znamke, je prikazan v kodi 6.

Izsek kode 6: Ujemanje znamke s dejanskim in približnim ujemanjem

```
1 def match_brand(raw: str, source: str):
```



```
2     normalized = normalize_text(raw)
3     brands = _get_all_brands()
4
5     for brand_id, canonical_name in brands:
6         if normalize_text(canonical_name) == normalized:
7             return brand_id, canonical_name
8
9     cached = _lookup_brand_alias(normalized, source)
10    if cached:
11        return cached
12
13    brand_names = [normalize_text(b[1]) for b in brands]
14    result = process.extractOne(normalized, brand_names,
15                               ↪ scorer=fuzz.WRatio)
16
17    if result is None or result[1] < 90:
18        print(f"[matching] No brand match for '{raw}', best score:
19              ↪ {result[1] if result else 0}")
20        return None
21
22    brand_id, canonical_name = brands[result[2]]
23    _store_brand_alias(normalized, brand_id, canonical_name, source)
24    return brand_id, canonical_name
```

S tem je prva faza s slike 4.8 zaključena: po normalizaciji so vsi podatki o oglasih vozil poenoteni v enotno strukturo (znamka, model, letnik, prevožena kilometrina, cena itd.), kar je predpogoj za primerjavo posameznih oglasov. Sledi druga faza, ocena tržne vrednosti. Njen prvi korak je sprejeti vhod uporabnika: implementirali smo končno točko API `/value`, ki sprejme štiri ključne attribute vozila iz poglavja 3.1.1 (znamko, model, letnik in kilometrino). V tej fazi smo se namenoma omejili zgolj nanje, saj je bil primarni cilj dokazati, da algoritem deluje na konceptualni ravni (angl. *proof of concept*), preden bi ga razširili na dodatne spremenljivke (npr. opremljenost, vrsto goriva, menjalnik, stanje vozila): dodajanje parametrov brez zadostne količine normaliziranih podatkov bi vzorec primerljivih vozil dodatno skrčilo, zaradi česar bi jih hitro ostalo premalo za smiselno statistično oceno. Po svoji zasnovi algoritem ustreza pristopu primerljivih prodaj (angl. *comparable sales approach*), ki je uveljavljena metoda vrednotenja nepremičnin in

vozil [31]. Cene predmeta pri tem pristopu ne ocenjujemo na podlagi enega samega primerka, temveč na podlagi množice podobnih, dejansko prodanih (oz. oglaševanih) primerkov, ki jim glede na stopnjo podobnosti pripišemo različno utež.

Rezultat algoritma pri tem ni zgolj ena sama cena, temveč cenovni razpon z dodatnimi pokazatelji kakovosti ocene; podrobneje ga opišemo pri zadnjem koraku v poglavju 4.7.4.

4.7.2 Filtriranje kandidatov iz podatkovne baze

Tukaj je opisan korak iskanja primerljivih vozil s slike 4.8. Izvede se s poizvedbo SQL, ki iz tabele `NormalizedCarListing` izlušči kandidate za primerjavo. Znamko in model filtriramo strogo (po enakosti), saj najmočnejše določata segment in bazno ceno vozila.

Numerični spremenljivki pa filtriramo z dovoljenim odstopanjem: pri kilometrini znaša $\pm 40\,000$ km (konstanta `KILOMETERS_DIFFERENCE`), pri letniku pa ± 3 leta (konstanta `YEAR_DIFFERENCE`). Navedeni meji nista bili izbrani naključno, temveč predstavljata kompromis med natančnostjo in velikostjo vzorca: ožje meje dajo bolj primerljiva vozila, a manjši vzorec in s tem večjo statistično negotovost – znano razmerje med pristranskostjo in varianco (angl. *bias-variance trade-off*) [27]. Ker je podatkovna baza normaliziranih podatkov v tej fazi razvoja še razmeroma majhna, smo se zavestno odločili za širši interval. Občutljivost ocene na te meje podrobneje preverimo v poglavju 5.6.

4.7.3 Točkovanje podobnosti posameznega vozila

Tukaj opišemo korak uteževanja podobnosti s slike 4.8. Za vsako vozilo iz filtrirane množice izračunamo oceno podobnosti (angl. *score*) glede na ocenjevano vozilo. Uporabili smo dve delni oceni (prvo za kilometrino in drugo za letnik), ki ju nato združimo v skupno oceno. Koda 7 predstavlja ta del izračuna.

Izsek kode 7: Izračun ocene podobnosti posameznega vozila

```
1 def get_scores_of_each_car(data_compare: dict, cars: list) -> list[tuple]:
2     results = []
3     for car in cars:
4         absolute_dif = abs(int(car["kilometers"]) -
5                               ↪ int(data_compare["kilometers"]))
6         kilometers_score = max(0, 100 * (1 - absolute_dif /
7                               ↪ KILOMETERS_DIFFERENCE))
8
9         absolute_dif = abs(int(car["registration_year"]) -
10                               ↪ int(data_compare["year"]))
11         registration_year_score = max(0, 100 * (1 - absolute_dif /
12                               ↪ YEAR_DIFFERENCE))
13
14         total_score = kilometers_score * 0.6 + registration_year_score *
15                               ↪ 0.4
16         results.append((float(car["price"]), total_score))
17     return results
```

Obe delni oceni sta zasnovani po istem načelu: vozilo, ki je identično ocenjevanemu, prejme največ 100 točk, vozilo na robu dovoljenega intervala (z odstopanjem natanko 40 000 km ali 3 leta) pa 0 točk. Med tema skrajnostma ocena linearno pada, funkcija `max(0, ...)` zagotavlja, da ocena nikoli ne postane negativna. Ker vozila zunaj dovoljenega intervala zaradi filtriranja v poizvedbi SQL sploh ne pridejo do tega koraka, gre pri tem predvsem za dodatno varnostno mejo.

Za linearno padajočo funkcijo (namesto eksponentne) smo se odločili iz dveh razlogov. Najprej zaradi enostavnosti in jasnosti, saj je linearna funkcija bolj razumljiva in preverljiva, kar je za prikaz delovanja koncepta pomembnejše od mogočega izboljšanja natančnosti. Drugi razlog je enostavno umerjanje, saj z eno samo konstanto natančno določimo, kje ocena doseže ničlo.

Skupno oceno podobnosti izračunamo kot uteženo vsoto obeh delnih ocen, pri čemer kilometrini pripišemo višjo utež (60 %) kot letniku (40 %). Vozili istega letnika se tako bolj razlikujeta po dejanski obrabi, če je prvo prevozilo 50 000 km, drugo pa 200 000 km. Vozili s podobno kilometrino, a nekoliko različnim letnikom pa ostajata bolj primerljivi.

4.7.4 Izračun pričakovane prodajne cene z uteženim povprečjem

Tukaj sta opisana zadnja koraka druge faze s slike 4.8, in sicer izračun ocene in njen rezultat. Ko vsa primerljiva vozila ovrednotimo z oceno podobnosti, končno pričakovano prodajno ceno izračunamo kot uteženo povprečje njihovih cen, pri čemer je utež vsakega vozila prav njegova izračunana ocena podobnosti:

$$\text{estimated_price} = \frac{\sum_i \text{price}_i \cdot \text{score}_i}{\sum_i \text{score}_i} \quad (4.1)$$

Uporaba uteženega povprečja namesto navadnega aritmetičnega je ključna: če bi izračunali navadno povprečje cen vseh vozil znotraj intervala, bi vozilo z roba intervala (39 000 več km) prispevalo enako kot vozilo, ki ima le 500 več km. To je neustrezno, saj je slednje veliko boljši pokazatelj dejanske cene. Z uteževanjem po vrednosti `total_score` pa bolj podobna vozila sorazmerno bolj vplivajo na končno oceno, vozila na robu intervala pa prispevajo le malo. Standardna oblika uteženega povprečja (angl. *weighted mean*) poskrbi, da vsota uteži v imenovalcu pravilno določi rezultat, ne glede na število vozil ali velikost njihovih ocen.

Poleg pričakovane prodajne cene algoritem vrne tudi stopnjo zaupanja (angl. *confidence*), ki je izračunana po enačbi:

$$\text{confidence} = \min\left(1.0, \frac{n}{20}\right), \quad (4.2)$$

pri čemer je n število vozil, najdenih znotraj filtriranih meja. To pove uporabniku stopnjo zaupanja v ceno, ki smo mu jo posredovali. Če je število primerljivih vozil veliko, je stopnja zaupanja bližje 1, sicer pa je stopnja zaupanja bližje 0. Tudi tu je vrednost 20 določena heuristično in bi jo lahko spremenili.

Rezultat algoritma tako ni zgolj ena sama cena. Končna točka `/value` vrne pričakovano prodajno ceno (`estimatedPrice`), poleg nje pa še dobro ceno (`goodPrice`) kot spodnjo mejo in slabo ceno (`badPrice`) kot zgornjo

mejo ter iz njiju izpeljano razmerje **ratio**, ki pove, kako razpršene so cene primerljivih vozil na trgu. Pri tem sta pomembni dve podrobnosti. Prvič, pri izračunani ceni upoštevamo popust, saj se rabljena vozila praviloma prodajo ceneje, kot so oglaševana. Drugič, razmerje **ratio** med **badPrice** in **goodPrice** je poleg stopnje zaupanja drugi pokazatelj kakovosti ocene: meri širino cenovnega razpona primerljivih vozil in s tem, koliko prostora ima uporabnik pri pogajanju.

4.8 Infrastruktura in avtomatizacija

Poleg implementacije posameznih komponent je bil pomemben del razvoja tudi vzpostavitev infrastrukture, ki zagotavlja konsistentno delovanje sistema v različnih fazah njegovega življenjskega cikla; od lokalnega razvoja, prek avtomatiziranega preverjanja kode, do namestitve v produkcijsko okolje. Namen tovrstne avtomatizacije je zmanjšati število ročnih posegov, ki so podvrženi napakam, in zagotoviti, da se aplikacija v vseh okoljih obnaša enako, ne glede na razlike v konfiguraciji računalnika ali strežnika. Tak pristop temelji na načelu usklajenosti okolij (angl. *environment parity*), po katerem naj bodo razvojno, testno in produkcijsko okolje čim bolj podobna, s čimer zmanjšamo tveganje za napake, ki se pojavijo šele ob namestitvi [69].

4.8.1 Postavitev razvojnega okolja z vsebniki

Za postavitev razvojnega okolja v vsebnikih smo uporabili orodje Docker Compose [15], ki omogoča enostavnejšo vzpostavitev več med seboj povezanih vsebnikov. Orodje deluje tako, da v datoteki `docker-compose.yml` opišemo vse potrebne storitve in njihove medsebojne odvisnosti, kar omogoča zagon celotnega okolja z enim samim ukazom. To je še posebej uporabno pri razvoju in testiranju. Za lokalno preizkušanje pošiljanja elektronske pošte smo uporabili orodje MailHog [39], ki odhodno pošto prestreže in jo prikaže v spletnem vmesniku, brez dejanskega pošiljanja na pravi naslov. Za zaledni sistem in sistem za vrednotenje vozil smo napisali ločeni datoteki `Dockerfile`,

v katerih določimo jezik in okolje sistema ter ukaze, ki naj jih vsebnik ob zagonu izvede.

Ker se storitve med seboj zaganjajo z različnimi hitrostmi, smo za pravilno zaporedje zagona uporabili `depends_on` v kombinaciji z definicijami `healthcheck`. Brez tega bi se zaledni sistem lahko poskusil prezgodaj povezati s podatkovno bazo, kar bi ob zagonu okolja povzročilo napako. Odvisne storitve zato vsebujejo ukaz za preverjanje pripravljenosti (`pg_isready` pri sistemu PostgreSQL oziroma `redis-cli ping` pri sistemu Redis [57]) in se zaženejo šele, ko je preverjanje uspešno.

Za shranjevanje podatkov smo uporabili poimenovane nosilce podatkov (angl. *named volumes*): prvi skrbi za trajnost podatkovne baze med ponovnimi zagoni vsebnikov, drugi pa predpomni mapo `node_modules` znotraj zalednega vsebnika, saj bi jo sicer prepisala mapa gostitelja. Za osveževanje ob spremembah (angl. *hot reload*), uporabljamo prevezavo map (angl. *bind mount*) iz gostiteljevega datotečnega sistema v vsebnik, tako da se spremembe v izvorni kodi takoj odražajo v delovanju aplikacije.

Odjemalskega sistema v lokalno razvojno okolje namenoma nismo vključili, saj njegova vključitev zaradi nenehnih ponovnih izgradenj slike ob vsaki spremembi bistveno upočasni razvoj. Med razvojem ga tako poganjamo neposredno na gostitelju.

Za produkcijsko okolje uporabljamo ločeno datoteko `docker-compose.prod.yml`, ki namesto lokalne gradnje slik uporablja vnaprej zgrajene slike iz registra vsebnikov (angl. *container registry*). Storitve poveže v skupno omrežje ter doda poimenovan nosilec podatkov za shranjevanje naloženih datotek.

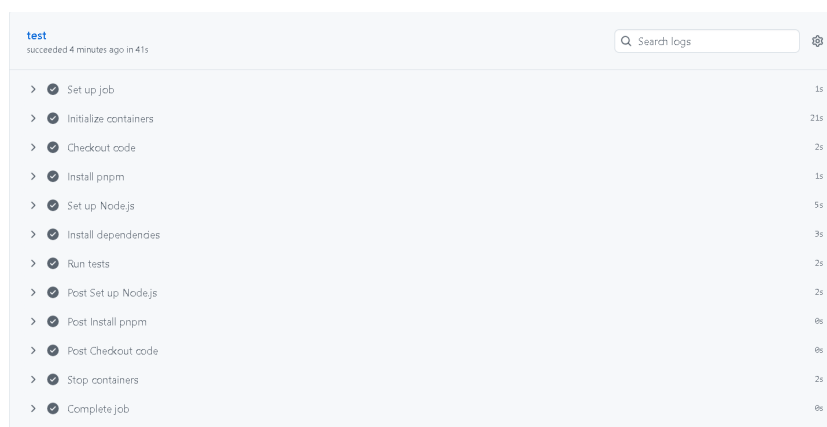
4.8.2 Neprekinjena integracija

Neprekinjena integracija (angl. *continuous integration*) je praksa, pri kateri ob vsaki spremembi kode samodejno stečejo preverjanja (testi, izgradnja, statična analiza). Tako napake odkrijemo, še preden se zlijejo v glavno vejo ali dosežejo druge razvijalce [21]: namesto da bi razvijalec pred vsako združitvijo ročno pognal teste, jih strežnik izvede sam, vsakič na enak način.

Za implementacijo neprekinjene integracije smo uporabili orodje GitHub Actions [25]. Njegovo delovanje opredeljujejo naslednji pojmi:

- **Potek dela** (angl. *workflow*) je določen v datoteki `.yaml` v mapi `.github/workflows/`, ki določa, kdaj naj se (npr. ob dogodku `pull_request` ali `push`) sprožijo določena opravila.
- **Opravilo** (angl. *job*) se izvede na svežem virtualnem strežniku (npr. `ubuntu-latest`), ki ga GitHub ob zagonu ustvari in po zaključku izbriše.
- **Koraki** (angl. *steps*) znotraj opravila so zaporedni ukazi. To so lahko neposredni lupinski ukazi (npr. `run: pnpm test`) ali vnaprej pripravljeni in ponovno uporabni gradniki (angl. *actions*), ki jih razvije GitHub ali skupnost. Tako gradnik `actions/checkout` prenese izvorno kodo, gradnik `pnpm/action-setup` pa namesti upravitelja paketov. Namesto lastnoročno napisane skripte (za namestitev okolja Node, predpomenjenje odvisnosti in nastavitev spremenljivke `PATH`) tako uporabimo že obstoječi gradnik.
- **Storitve** (angl. *services*) so pomožni vsebniki, ki tečejo vzporedno z opravilom. Uporabni so takrat, ko testi potrebujejo pravo podatkovno bazo namesto njene simulacije (angl. *mock*).

Potek dela v datoteki `ci.yaml` smo določili tako, da se ob vsakem novem zahtevku za združitev (angl. *pull request*) izvedejo vsi testi zalednega sistema; ob tem se preveri, ali se uspešno izvedejo. Potek dela v datoteki `docker-publish.yaml` je zasnovan še korak dlje: ob zahtevku za združitev se zgolj preveri, ali se slika Docker uspešno zgradi, ob združitvi v glavno vejo pa se slika tudi dejansko zgradi, digitalno podpiše in objavi. Ta korak že sodi na področje neprekinjene dostave (angl. *continuous delivery*), torej samodejne distribucije zgrajene programske rešitve [29]. Na naslednji sliki je prikazan primer izgradnje in izvajanja testov na cevovodu (slika 4.9).



Slika 4.9: Samodejno izvajanje testov nad vejo na cevovodu

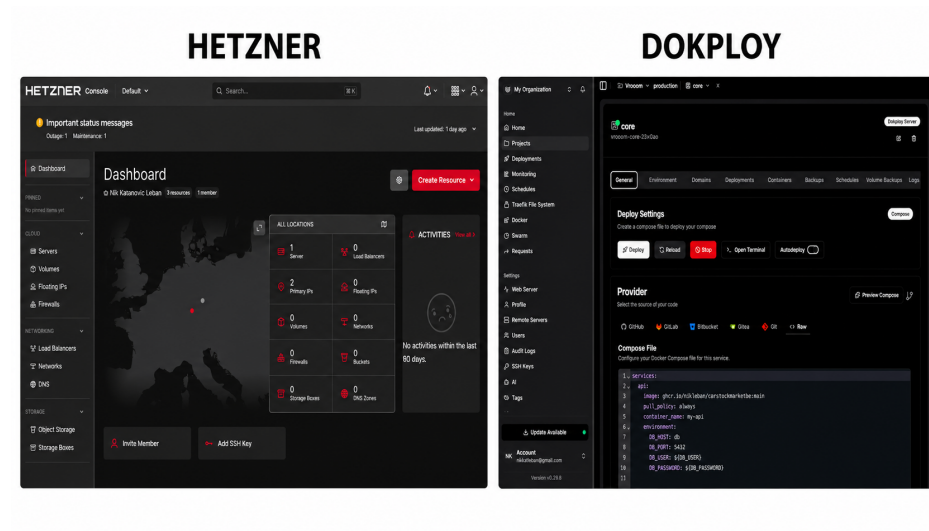
4.8.3 Namestitev in gostovanje

Za gostovanje smo izbrali eno izmed cenovno najugodnejših možnosti, in sicer najeti navidezni zasebni strežnik (angl. *virtual private server*) pri ponudniku Hetzner [28]. Strežnik razpolaga s 4 GB delovnega pomnilnika in dvema procesorskima jedroma, kar zadostuje za zagon vseh treh storitev. Po prvi prijavi v strežnik smo najprej posodobili operacijski sistem na najnovejšo različico, nato pa namestili orodji Docker in Docker Compose.

Za upravljanje aplikacije smo uporabili orodje Dokploy [16], ki omogoča enostavno uporabo datotek `docker-compose` v produkcijskem okolju in hkrati močno poenostavi postopek posodabljanja aplikacije. V njem lahko nastavimo tudi opravila, ki se izvajajo periodično (podobno kot storitev `cron`). Na strežnik je treba namestiti še povezovalno komponento med orodjema Dokploy in Docker, nato pa Dokploy sam poskrbi za izgradnjo aplikacije: iz repozitorija GitHub zajame najnovejšo različico kode in zgradi pripadajočo sliko neposredno na strežniku. Tak postopek omogoča, da se vsaka sprememba v produkcijskem okolju odrazi v nekaj minutah.

Dokploy je odprtokodno in brezplačno orodje, najem strežnika pri ponudniku Hetzner pa predstavlja nizek mesečni strošek (trenutno 6,50 € na mesec). Levo na sliki 4.10 je upravljalna stran, desno pa je uporabniški vmesnik Do-

kploy.



Slika 4.10: Upravljalna stran Hetzner in Dokploy

Poglavje 5

Analiza in vrednotenje rešitve

V tem poglavju sistematično pregledamo in analiziramo razvito rešitev. Najprej iz zahtev in izhodišč, ki smo si jih zastavili na začetku, izpeljemo raziskovalna vprašanja in za vsako določimo merilo uspešnosti (poglavje 5.1). Nato opišemo merilno okolje in postopke (poglavje 5.2) ter natančno definiramo vse uporabljene mere (poglavje 5.3). V poglavjih 5.4–5.8 sledijo rezultati: značilnosti zbranih podatkov, natančnost ocene vrednosti, občutljivost algoritma na parametre, latenca posameznega vrednotenja in obnašanje pod obremenitvijo. Poglavje sklenemo s preverjanjem nefunkcionalnih zahtev (poglavje 5.9), preverjanjem funkcionalnih zahtev skupaj z uporabniškim preizkusom (poglavje 5.10) in simulacijo uporabe (poglavje 5.11).

5.1 Raziskovalna vprašanja in merila uspešnosti

MdAPE – *Mediana absolutne odstotne napake* je osrednja mera natančnosti v tem poglavju; nanjo se sklicujejo merila uspešnosti in vsa nadaljnja poglavja. Izračunamo jo tako, da za vsak testni primer določimo absolutno odstotno napako, torej za koliko odstotkov se pričakovana prodajna cena razlikuje od dejanske, nato pa čez vse primere vzamemo mediano teh vrednosti. $\text{MdAPE} = 15\%$ tako pomeni, da je pri polovici vozil ocena znotraj 15%

od dejanske cene, pri drugi polovici pa je odstopanje večje; manjša kot je vrednost, natančnejše so ocene. Mediano uporabljamo namesto povprečja, ker je manj občutljiva na posamezne skrajne napake. Natančne definicije MdAPE in preostalih mer so v poglavju 5.3.

Iz izhodišča, ki smo si ga zastavili v poglavju 1 (*uporabna in dovolj natančna* ocena tržne vrednosti, ki izboljša uporabniško izkušnjo), in iz nefunkcionalnih zahtev (poglavje 4.1) izpeljemo štiri raziskovalna vprašanja (RV1–RV4) in za vsako določimo merilo uspešnosti. Raziskovalna vprašanja so:

- **RV1 – natančnost ocene.** Kako natančno sistem oceni tržno vrednost vozila v primerjavi z dejanskimi oglasi na trgu? *Merilo:* MdAPE je pri vozilih z vsaj 10 primerljivimi vozili manjša od 15 %, ocena pa je bistveno natančnejša od preproste (naivne) referenčne metode.
- **RV2 – odvisnost natančnosti od podatkov.** Od česa je natančnost ocene najbolj odvisna (velikost vzorca primerljivih vozil, razpršenost ponudbe, segment vozila)?
- **RV3 – hitrost in razširljivost.** Ali sistem zadošča nefunkcionalnim zahtevam glede odzivnosti tudi pod sočasno obremenitvijo, in kje je zgornja meja trenutne postavitve?
- **RV4 – uporabnost.** Ali ciljni uporabniki znajo brez navodil opraviti glavne naloge (registracija, objava oglasa, vrednotenje)?

5.2 Metodologija in merilno okolje

Merilno okolje. Meritve hitrosti (poglavji 5.7 in 5.8) so izvedene v *produkcijskem* okolju, torej na navideznem zasebnem strežniku z dvema procesorskima jedroma in 4 GB pomnilnika ter s tehnologijami in njihovimi različicami, opisanimi v poglavju 4, natančneje v 4.8.3. Analitične in merilne skripte tečejo nad isto podatkovno bazo kot na produkciji.

Posnetek podatkov. Meritve v tem poglavju temeljijo na posnetku podatkovne baze iz septembra 2026 (njegove značilnosti so v poglavju 5.4). Ker se posnetek med analizo ne spreminja, so rezultati ponovljivi.

Orodja za meritve in analizo v tem poglavju smo napisali namenske skripte, ločene od produkcijske kode; ena od njih je dokumentirana kopija ocenjevalnega algoritma, parametrizirana za eksperimente. Statistično obdelavo in izris grafov smo izvedli s knjižnicama `NumPy` [26] in `Matplotlib` [30].

Natančnost ocene (poglavje 5.5) merimo z naključno izbranim normaliziranim oglasom: njegove ključne attribute uporabimo kot vhod v algoritem, sam oglas pa izločimo iz primerjalne množice. Nato izračunamo oceno in jo primerjamo z *dejansko oglaševano* ceno tega oglasa. Postopek ponovimo za 3000 naključnih oglasov.

5.3 Uporabljene mere

Latenca je čas, ki preteče od trenutka tik pred oddajo zahtevka do trenutka, ko odjemalec prejme celoten odgovor. Glede na to, kolikšen del te poti zajamemo, ločimo tri vrste:

- (a) *latenca obdelave na strežniku* – čas izvajanja funkcije za vrednotenje, od začetka do konca, brez omrežja;
- (b) *latenca poizvedbe SQL* – čas izvajanja poizvedbe v podatkovni bazi, od začetka do konca, brez omrežja;
- (c) *latenca od konca do konca* (angl. *end-to-end*) – (a) skupaj z omrežnim obhodom med odjemalcem in strežnikom ter obdelavo zahtevka HTTP; to je čas, ki ga zazna uporabnik.

V poglavju 5.7 merimo vrsti (a) in (b), v poglavju 5.8 pa vrsto (c).

Percentili. Če meritve uredimo po velikosti, je p_k (percentil k) tista vrednost, od katere je k odstotkov meritev manjših ali enakih. Tako p_{95} latence 200 ms pomeni, da je 95 % zahtevkov obravnavanih v 200 ms ali hitreje, preostalih 5 % pa počasneje. Poročamo p_{50} (mediana), p_{95} in p_{99} .

Prepustnost, sočasnost, nasičenje. Obnašanje sistema pod obremenitvijo opišemo s tremi količinami:

- **prepustnost** – število uspešno obdelanih zahtevkov na sekundo;
- **sočasnost** – število odjemalcev, ki hkrati držijo odprt zahtevek;
- **nasičenje** (*koleno*) – točka, v kateri dodatna sočasnost ne poveča več prepustnosti, latenca pa začne naraščati približno linearno.

Pokritost je delež poizvedb, za katere sistem vrne oceno, torej za katere najde vsaj eno primerljivo vozilo.

Raznolikost ponudbe (v izvorni kodi `ratio`) je relativna širina cenovnega razpona primerljivih vozil, kjer sta P_{25} in P_{75} (v kodi `goodPrice` in `badPrice`) uteženi 25. oziroma 75. percentil cen primerljivih vozil, $\hat{p}$ (v kodi `estimatedPrice`) pa pričakovana prodajna cena vozila:

$$\text{raznolikost} = \frac{P_{75} - P_{25}}{\hat{p}}. \quad (5.1)$$

Vrednost izražamo v odstotkih; enoten zapis uporabljamo v vseh tabelah tega poglavja.

5.4 Značilnosti zbranih podatkov

Viri in obseg. Velika večina oglasov je bila zajeta z največjega slovenskega spletnega portala za prodajo rabljenih vozil (poglavje 3.2); manjši del predstavlja vzorec z večjega nemškega oglašnega portala. Zajem je zajel javno dostopne oglase za nekomercialno raziskovalno uporabo, z omejeno hitrostjo

zahtevkov in brez shranjevanja kontaktnih ali osebnih podatkov prodajalcev. Po normalizaciji baza obsega 85.846 oglasov (83.493 s slovenskega in 2.353 z nemškega portala) za 10 znamk vozil; razčlenitev po znamki je v tabeli 5.1.

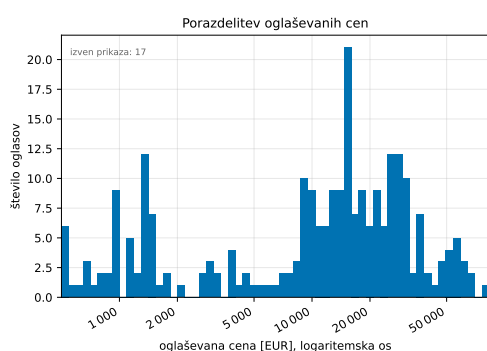
Tabela 5.1: Število normaliziranih oglasov po znamki.

Znamka	Oglasi	Znamka	Oglasi
Volkswagen	21.191	Peugeot	7.513
Audi	11.842	Citroën	6.729
BMW	10.724	Ford	5.783
Renault	9.615	Toyota	2.648
Mercedes-Benz	9.355	Alfa Romeo	446

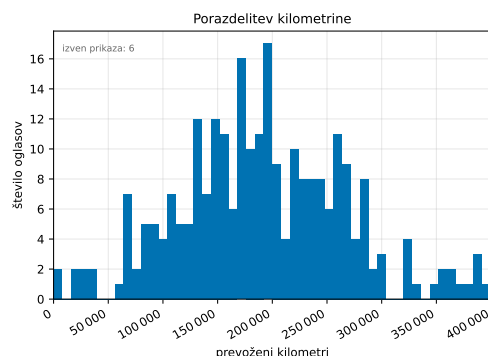
Porazdelitve ključnih atributov. Slika 5.1 prikazuje porazdelitve oglaševanih cen, prevožene kilometrine in letnika. Porazdelitev cen je izrazito desno asimetrična, zato jo prikazujemo na logaritemski osi. Mediana cene je 14.890 EUR (povprečje 18.137 EUR), mediana kilometrine 159.950 km, mediana letnika pa 2017; podrobnosti so v tabeli 5.2. Njihov razpon pa je razviden iz stolpca *maks.* v tabeli 5.2 in iz obravnave osamelcev v tabeli 5.3.

Tabela 5.2: Povzetek porazdelitev ključnih numeričnih atributov.

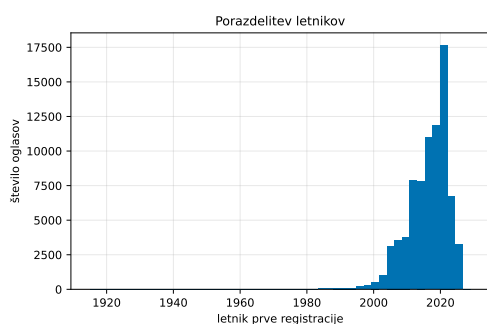
	p_5	mediana	povprečje	p_{95}	maks.
cena [EUR]	1.550	14.890	18.137	49.014	374.000
kilometrina [km]	26.000	159.950	169.464	334.000	1.974.001
letnik	2005	2017	2016	2024	2029
moč [kW]	56	110	117	221	920



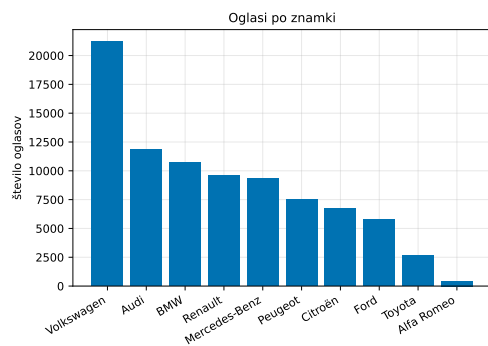
(a) oglaševana cena (log)



(b) prevoženi kilometri



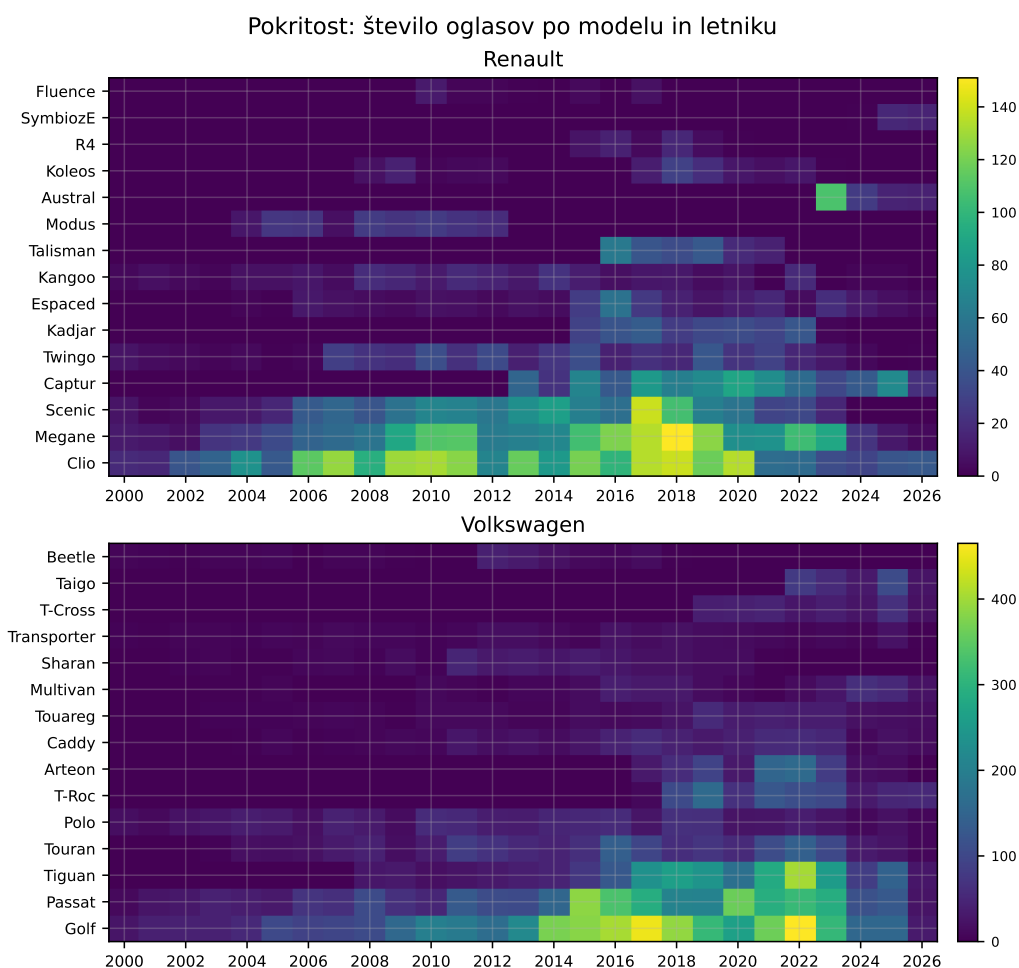
(c) letnik prve registracije



(d) oglasi po znamki

Slika 5.1: Porazdelitve ključnih atributov oglasov

Pokritost po segmentih. Za pošteno oceno mora imeti algoritem dovolj primerljivih vozil v ožjem segmentu (ista znamka in model, podoben letnik in kilometrina). Slika 5.2 prikazuje toplotno karto za znamki Renault in Volkswagen: vrstice so modeli, stolpci pa letniki; število oglasov je označeno z barvo, pri čemer toplejša barva pomeni boljšo pokritost. Vidno je, da so pogosti modeli srednjih letnikov (npr. Volkswagen Golf ali Renault Clio) dobro pokriti, redki modeli in skrajni letniki pa slabše, kar neposredno pojasni razlike v zanesljivosti ocen (poglavje 5.5).



Slika 5.2: Pokritost za znamki Renault in Volkswagen (po ena znamka na vrstico).

Osamelci in kakovost podatkov. Postopek normalizacije trenutno ne izvaja filtriranja osamelcev, zato baza vsebuje tudi vozila z napačnimi vrednostmi ali pa z ekstremi. Tabela 5.3 prikazuje njihov delež. Kar 10,7% oglasov ima vsaj eno vrednost zunaj smiselnih mej, največ pri kilometrini (9,9%; najvišja zabeležena vrednost je skoraj dva milijona kilometrov) in letniku (8,1%; nekaj oglasov ima celo letnik v prihodnosti).

Tabela 5.3: Osamelci v normalizirani bazi (85.846 oglasov).

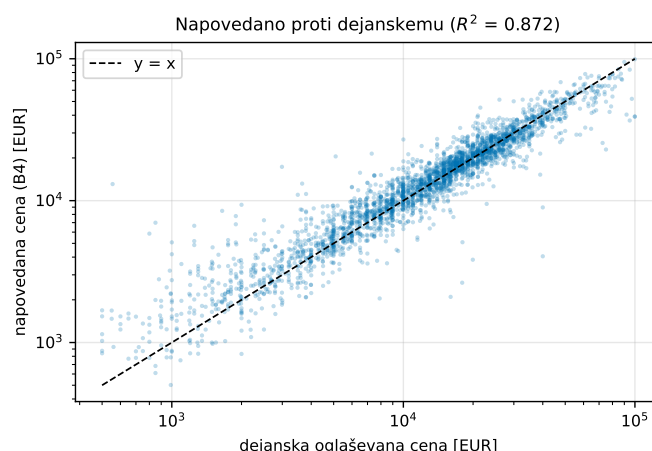
Merilo	Št. vrstic	Delež
cena zunaj [500, 200.000] EUR	427	0,5 %
kilometrina zunaj [1000, 500.000] km	8.495	9,9 %
letnik zunaj [1990, 2027]	6.979	8,1 %
vsaj ena vrednost zunaj mej	9.169	10,7 %

5.5 Natančnost ocene vrednosti

To je najpomembnejša meritev, saj neposredno preverja zastavljeno vprašanje o *dovolj natančni* oceni (RV1). Protokol je opisan v poglavju 5.2, rezultate pa izdela skripta `analysis/accuracy_cv.py` nad vzorcem 3000 oglasov (od tega jih je 2986 dobilo oceno).

Merilo resnice in namerni odmik ocene. Kot merilo resnice uporabljamo *oglaševano* ceno, saj dejanske prodajne cene niso javno dostopne. Sistem pa namenoma ne ocenjuje oglaševane cene, temveč *pričakovano prodajno ceno*: ker se rabljena vozila v pogajanjih praviloma prodajo pod oglaševano ceno, oceno zniža s stopenjskim popustom (9–15 % glede na cenovni razred). Zato je pri primerjavi z oglaševano ceno pričakovan rahel *negativni* odmik ocene, kar je zaželeno in ne pomeni slabše ocene.

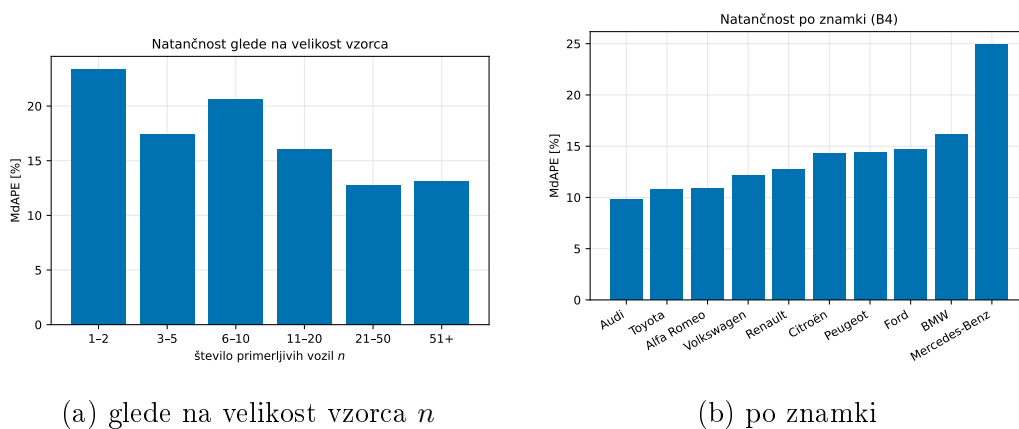
Skupni rezultat. Slika 5.3 to potrjuje vizualno: pike se v celotnem cenovnem razponu tesno oprijemajo idealne premice $y = x$ brez sistematičnega upogiba, kar pomeni, da ocena ni pristranska glede na cenovni razred ($R^2 = 0,87$). Ker je razpršenost v logaritemskem merilu enakomerna, je napaka podobna pri cenejših in dražjih vozilih. Izjema je najcenejši del pod približno 2000 EUR, kjer je primerljivih vozil malo, pike so bolj raztresene in ležijo pretežno nad premico – model tam ceno pogosteje preceni.



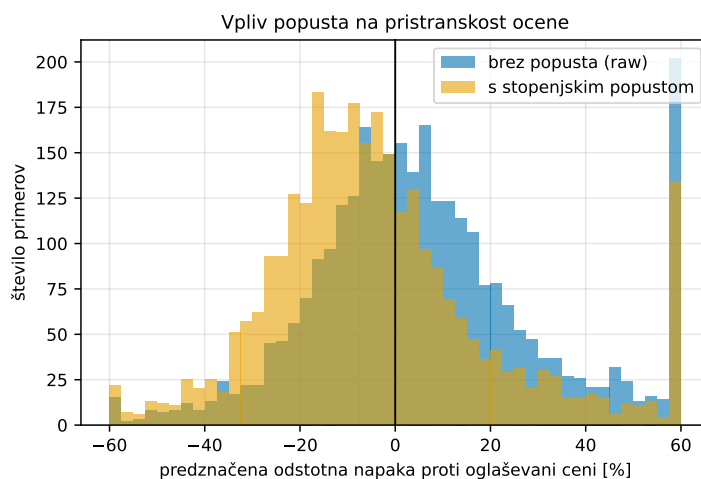
Slika 5.3: Pričakovana prodajna cena proti oglaševani (leave-one-out, brez popusta).

Odvisnost od velikosti vzorca in segmenta (RV2). Natančnost je odvisna predvsem od tega, koliko primerljivih vozil je v segmentu. Napaka pada z velikostjo vzorca in se pri $n \geq 20$ ustali pri približno 13 % MdAPE (slika 5.4a); to podpira izbiro konstante 20 v stopnji zaupanja (enačba 4.2). Po segmentih napaka (MdAPE) narašča s starostjo in kilometrino, najmočnejše pa z *raznolikostjo ponudbe*: od 7,5 % pri modelih z enotno ponudbo do 29,7 % pri razpršeni. Po znamki (slika 5.4b) so najnatančnejši pogosti, gosto pokriti modeli, najmanj pa Mercedes-Benz (24,9 %), pri katerem normalizacija zlije več modelov v isto ime in poslabša izbor primerljivih vozil.

Vpliv popusta. Slika 5.5 prikazuje predznačeno napako s popustom in brez njega. Brez popusta je ocena rahlo optimistična (mediana +3,8 % nad oglaševano ceno), z namernim popustom pa se premakne na -6,8 % pod oglaševano ceno.



Slika 5.4: Odvisnost natančnosti ocene od podatkov.



Slika 5.5: Predznačena napaka proti oglaševani ceni, s popustom in brez njega.

5.6 Občutljivost algoritma na parametre

Algoritem ima več hevristično določenih konstant: dovoljeno odstopanje kilometrine in letnika pri iskanju primerljivih vozil, uteži delnih ocen in stopenjski popust. Da preverimo, kako močno je rezultat odvisen od njihove izbire, vsako od njih po vrsti spreminjamo v razumnem razponu, ostale pa pustimo

pri privzetih vrednostih, in opazujemo MdAPE ter pokritost.

Na večino konstant je algoritem neobčutljiv: pri spreminjanju okna kilometrine in razmerja uteži ostane MdAPE med 13,2 in 13,6 %. Edini parameter z opaznim vplivom je širina okna letnika (tabela 5.4): pri ± 1 letu je MdAPE 12,6 %, pri ± 5 letih pa 15,2 % – ožje okno da bolj primerljiva vozila, a manjši vzorec. Sedanja izbira ± 3 leta je kompromis med natančnostjo in pokritostjo. Stolpec *mediana n* pove, koliko primerljivih vozil ima poizvedba pri dani širini okna. Število primerljivih vozil je namreč za vsako znamko drugačno in močno odvisno od modela, zato ne moremo navesti enega samega n , temveč vzamemo mediano čez vseh ~ 3000 poizvedb v vzorcu – polovica jih ima manj, polovica več primerljivih vozil od navedene vrednosti. Razširitev okna to število strmo poveča, MdAPE pa pusti tako rekoč nespremenjen: dodatna, manj podobna vozila ne izboljšajo ocene, povečajo le pokritost.

Tabela 5.4: Občutljivost na širino okna primerljivosti. Pokritost je delež poizvedb z vsaj 5 primerljivimi vozili.

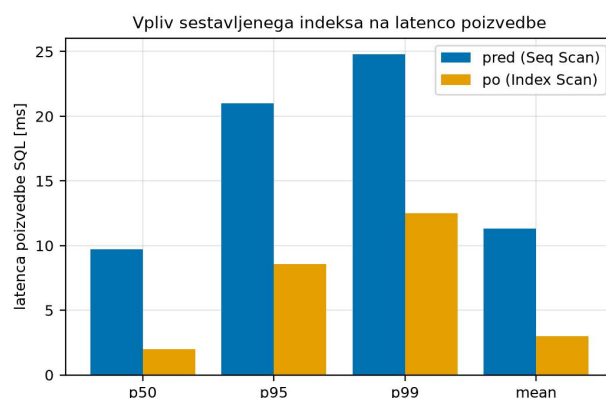
Parameter	MdAPE [%]	Pokritost [%]	Mediana n
kilometrina ± 10.000 km	13,2	92,3	53
kilometrina ± 40.000 km	13,4	98,1	196
kilometrina ± 80.000 km	13,6	99,0	362
letnik ± 1 leto	12,6	95,6	97
letnik ± 3 leta	13,4	98,1	196
letnik ± 5 let	15,2	98,5	262

Algoritem je torej robusten: njegova natančnost je odvisna predvsem od gostote in kakovosti podatkov (poglavje 5.5), ne od finega nastavljanja parametrov.

5.7 Latenca posameznega vrednotenja

Latenco posameznega vrednotenja izmerimo z 2000 neposrednimi klici funkcije nad naključnimi realnimi vhodi (brez HTTP, po 100 ogrevalnih klicih; `benchmarks/bench_algo.py`), pri čemer ločeno beležimo čas poizvedbe SQL, točkovanja v Pythonu in serializacije odgovora.

Vpliv sestavljenega indeksa. Prvotna poizvedba ni imela ustreznega indeksa, zato je pri vsakem vrednotenju prebrala celotno tabelo. Čas je zaradi tega rasel linearno z njeno velikostjo. Po dodajanju sestavljenega indeksa na (`brand_name`, `model_name`, `registration_year`, `kilometers`) je mediana latence poizvedbe SQL padla z 9,7 ms na 2,0 ms (slika 5.6). Bistveno je, da čas poizvedbe s tem postane neodvisen od velikosti tabele – $O(\text{zadetki})$ namesto $O(\text{tabela})$.



Slika 5.6: Latenca poizvedbe SQL pred in po dodajanju sestavljenega indeksa.

Razčlenitev latence. Po dodajanju indeksa tipično vrednotenje traja 2,0 ms, v najslabšem odstotku primerov (p99) pa 13,5 ms (tabela 5.5). Skoraj vsa latenca odpade na poizvedbo SQL; točkovanje v Pythonu (0,4 ms povprečno) in serializacija odgovora sta zanemarljiva.

Vrednotenje se torej izvede daleč pod nefunkcionalno zahtevo 2 s (po-

Tabela 5.5: Latenca posameznega vrednotenja po dodajanju indeksa ($N = 2000$), v milisekundah.

Komponenta	p50	p95	p99	Povprečje
Skupaj	2,0	9,7	13,5	3,1
Poizvedba SQL	1,8	8,4	11,5	2,7
Točkovanje (Python)	0,2	1,2	1,8	0,4
Serializacija (JSON)	0,01	0,02	0,03	0,01

glavje 4.1), ob prisotnosti indeksa pa je dovolj prostora tudi za rast podatkovne baze.

5.8 Obnašanje pod obremenitvijo

Merimo end-to-end latenco (vrsta (c)) in prepustnost končne točke `POST /value` pri naraščajoči sočasnosti. Za obremenitev, smo ustvarili skripto z več vzporednimi odjemalci, ki v fiksnem 20-sekundnem oknu neprekinjeno pošiljajo zahteve z naključnimi realnimi vhodi.

Tabela 5.6: Prepustnost in latenca pri naraščajoči sočasnosti.

Sočasnost	Prep. [z/s]	p50 [ms]	p95 [ms]	p99 [ms]
1	161	5,0	13,8	17,5
2	197	8,9	19,6	24,6
5	218	21,7	41,3	51,6
10	220	43,0	75,7	95,9
20	225	84,4	144,4	177,0
40	198	170,8	413,7	870,7

Rezultat. Na nobeni stopnji ni bilo napak. Prepustnost naraste s 161 z/s pri sočasnosti 1 na približno 220 z/s že pri sočasnosti 5 in se nato ustali. Od

te točke naprej dodatna sočasnost ne poveča prepustnosti, temveč le podaljša latenco, ki raste približno linearno s sočasnostjo (mediana 22 ms pri 5, 84 ms pri 20, 171 ms pri 40). p_{95} ostane pod 0,5 s tudi pri najvišji preizkušeni sočasnosti, torej krepko pod nefunkcionalno zahtevo 2 s.

Ozko grlo. Prepustnost se ustavi pri ≈ 220 z/s, čeprav bi ob povprečnem vrednotenju 3,1 ms samo procesorska moč dopuščala ≈ 320 z/s. Ozko grlo torej ni računska moč, temveč to da promet teče skozi eno samo deljeno povezavo do baze (`app/database.py`) in en strežniški proces, ki zahtevke obdelujeta zaporedno. Zato dodatna sočasnost nad 5 ne poveča prepustnosti, temveč le sorazmerno podaljša latenco.

5.9 Preverjanje nefunkcionalnih zahtev

Tabela 5.7 za vsako nefunkcionalno zahtevo iz poglavja 4.1 navaja merilo, izmerjeno vrednost in sodbo.

Tabela 5.7: Preverjanje nefunkcionalnih zahtev.

Zahteva	Merilo	Izmerjeno	Sodba
Odziv vmesnika na dejanje uporabnika	< 100 ms	≈ 50 ms (na strani odjemalca)	OK
Zahtevki HTTP (razen vrednotenja)	< 300 ms	< 100 ms (p_{95} , ocena)	OK
Vrednotenje vozila	< 2 s	13,5 ms (p_{99} , brez omrežja)	OK
Vizualna privlačnost / dostopnost	Lighthouse	/	OK
Varovanje uporabniških podatkov	bcrypt, JWT z rokom	/	OK
Odzivnost pri zahtevnejših opravilih	asinhrona obdelava	/	OK

Odzivnost vmesnika in zahtevkov. Uporabniški vmesnik teče v brskalniku (React), zato se odziv na dejanje uporabnika izriše lokalno, brez obhoda do strežnika, in je praktično trenuten. Latenco zahtevkov HTTP določa predvsem omrežje: strežnik gostuje pri ponudniku Hetzner v Nemčiji, obhodni čas iz Slovenije je ≈ 25 ms, preproste končne točke API pa dodajo le nekaj milisekund poizvedbe po indeksu, tako da p_{95} ostane krepko pod zahtevo 300 ms.

5.10 Preverjanje funkcionalnih zahtev z uporabniškim preizkusom

Funkcionalne zahteve smo preverili z uporabniškim preizkusom: vsak udeleženec je izvedel scenarij, ki pokrije glavne funkcionalne zahteve iz poglavja 4.1 (registracija in prijava, objava oglasa, všečkanje, izbira jezika, vrednotenje vozila). Preizkus je bil namenjen odkrivanju težav uporabnosti.

Udeleženci in postopek. V preizkus smo vključili 6 udeležencev iz kroga družine in bližnjih prijateljev, katerih starost in računalniška pismenost sta se med seboj razlikovali. Vsak udeleženec je prejel enak seznam nalog (naštet zgoraj). Med izvajanjem je udeleženec razmišljal na glas, opazovalec pa je beležil, ali je nalogo dokončal samostojno in vsako opaženo težavo zapisal.

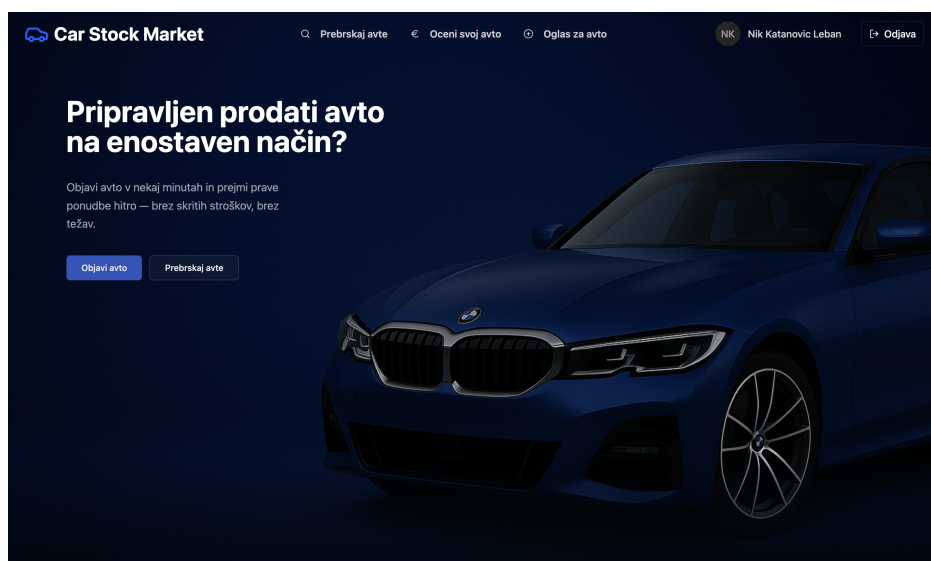
Rezultati. Vsi udeleženci so vse naloge dokončali brez pomoči: preverjane funkcionalne zahteve so torej izpolnjene, glavne poti v vmesniku pa dovolj jasne (RV4). Kljub temu so odkrili nekaj pomanjkljivosti; tabela 5.8 jih razvršča po resnosti (lestvica 0–4) in navaja, ali so bile odpravljene.

Tabela 5.8: Težave, odkrite med uporabniškim preizkusom (resnost po lestvici 0–4).

Težava	Resnost	Status
Premajhni naslovi podstrani v glavi glavne strani	2	odpravljeno
Manjkajoče polje za ponovni vnos gesla pri registraciji	4	odpravljeno
Velika števila brez ločil za tisočice	1	odpravljeno
Manjkajoč opis pod slikami oblik vozil v obrazcu	2	odpravljeno
Premalo vozil, dostopnih za vrednotenje	3	v načrtu

5.11 Simulacija uporabe

Delovanje rešitve ponazorimo s potekom uporabe. Uporabnik je ob vstopu usmerjen na glavno stran (slika 5.7), kjer sta izpostavljeni prijava in registracija ter pregled novih in vseh objavljenih oglasov. Registriran uporabnik lahko objavi oglas prek vodene forme (slika 5.8), ovrednoti poljubno vozilo, všečka oglase in si ogleda svoj profil z všečkanimi vozili (slika 5.9).



Slika 5.7: Glavna stran prototipa

Car Stock Market NK Nik Katanovic Leban Odjava

Ustvari oglas za avto

Vnesite podatke resnično in izpolnite obrazec. Začnimo!

- 1 Splošne informacije
- 2 Tehnične podrobnosti
- 3 Stanje in oprema
- 4 Fotografije in cena

Splošne informacije

Znamka:

Model:

Prva registracija (leto):

Prva registracija (mesec):

Prevoženih km: km

[Naprej](#)

Slika 5.8: Vodena forma za objavo oglasa

Car Stock Market Prebrskaj avte Oцени svoj avto Oglas za avto NK Nik Katanovic Leban Odjava

Nik Katanovic Leban [Preverjano](#)

[Uredi profil](#)

[Nastavitve](#)

4.8 Ocene 5 Aktivni oglasi 0 Prodani avti Član od 2026

Pregled Moji oglasi **Všečkani avti** Shranjeni filtri

Všečkani avti

Avti, ki ste jih shranili za pozneje

BMW 128i Convertible
32.321 €

2023 3.233 km
433 KM Hibrid

[Podrobnosti](#)

Hyundai Accent/Brio
32.322 €

2023 32.320 km
43 KM Dizel

[Podrobnosti](#)

Audi R8
23.230 €

2022 23.478 km
418 KM Bencin

[Podrobnosti](#)

Slika 5.9: Stran uporabnika s prikazanimi všečkanimi vozili

Primerjava ocen z obstoječimi portali. Za ilustracijo smo delovanje sistema primerjali s ponudbo na portalih avto.net in mobile.de na treh naključnih vozilih (tabela 5.9):

- Audi A3, letnik 2010, s 200 000 prevoženimi kilometri;
- BMW serije 2, letnik 2013, s 170 000 prevoženimi kilometri;
- Citroën Berlingo, letnik 2017, s 130 000 prevoženimi kilometri.

Tabela 5.9: Primerjava pričakovanih prodajnih cen s ponudbo na spletnih portalih (v evrih).

	Audi A3	BMW ser. 2	Citroën Berlingo
avto.net (razpon)	4.000–6.000	9.000–12.000	7.500–10.000
mobile.de (razpon)	4.500–9.000	pribl. 11.100	8.000–12.000
Sistem – dobra cena	4.500	7.600	7.600
Sistem – sredinska cena	6.350	9.300	9.941
Sistem – slaba cena	7.600	10.500	12.100
Raznolikost	49 %	31 %	45 %

Razpon med dobro in slabo oceno sledi izmerjeni raznolikosti ponudbe: najožji je pri BMW serije 2 (31 %), širši pa pri Citroën Berlingo (45 %) in Audi A3 (49 %). Sredinska ocena se najboljše ujema s ponudbo portalov pri Citroënu Berlingo (znotraj obeh razponov) in Audiju A3 (znotraj razpona **mobile.de**, tik nad zgornjo mejo **avto.net**), pri BMW serije 2 pa je nekoliko nižja od ponudbe na obeh portalih. Širši razpon med dobro in slabo oceno pomeni manj zanesljivo sredinsko oceno, kar je skladno z ugotovitvijo iz poglavja 5.5, da napaka narašča z raznolikostjo ponudbe. Pri interpretaciji je treba upoštevati, da sistem ocenjuje pričakovano prodajno in ne oglaševano ceno.

Poglavje 6

Sklep

Cilj diplomskega dela je bil razviti prototip spletne aplikacije na podlagi jasno opredeljenih zahtev, ki uporabnikom omogoča objavo in pregled oglasov rabljenih vozil, ter vanj vgraditi sistem za samodejno ocenjevanje tržne vrednosti vozila na podlagi primerjave z drugimi vozili na trgu. Oba cilja sta bila dosežena: rezultat je delujoč prototip aplikacije z ločeno odjemalsko in zaledno komponento ter komponento za vrednotenje vozil. Prototip aplikacije je trenutno nameščen v produkcijskem okolju in podpira vse zastavljene funkcionalne zahteve (registracija, prijava ter objava in pregled oglasov do vsečkanja in vrednotenja poljubnega vozila). Izpolnjene so tudi nefunkcionalne zahteve, ki smo jih v poglavju 5 sistematično preverili in izmerili.

Drugi cilj – ponuditi uporabno in dovolj natančno oceno tržne vrednosti – je bil prav tako dosežen. Uporabniški preizkus je potrdil, da ciljni uporabniki brez navodil opravijo vse glavne naloge, vmesnik pa je razumljiv in enostaven za uporabo. Glavne omejitve rešitve ostajajo odvisnost od enega prevladujočega podatkovnega vira, nižja zanesljivost ocene pri starejših vozilih in modelih z razpršeno ponudbo ter omejen obseg preizkusa z uporabniki.

- **Razširitev nabora vhodnih spremenljivk.** Ko bo na voljo dovolj normaliziranih podatkov, bi bilo smiselno v algoritem vključiti dodatne parametre (na primer opremo, vrsto goriva, tip menjalnika in stanje

vozila), kar bi izboljšalo natančnost ocene. S številom uporabnikov bo naraščal tudi obseg lastnih podatkov, zato odvisnost od zunanjih virov ne bo več potrebna.

- **Dinamično prilagajanje intervalov primerljivosti.** Namesto fiksnih mej bi lahko intervale kilometrine in letnika prilagajali sproti glede na gostoto podatkov v posameznem segmentu (na primer širši interval pri redkih modelih in ožji pri pogostih), s čimer bi omilili razmerje med pristranskostjo in varianco.
- **Izboljšanje kakovosti podatkov.** Analiza (poglavje 5.4) je pokazala, da približno 11 % normaliziranih oglasov vsebuje napačne oziroma ekstremne vrednosti, zato bi bilo smiselno v cevovod dodati samodejno filtriranje osamelcev.
- **Uvedba metod strojnega učenja in umetne inteligence za vrednotenje.** Sedanji pristop primerljivih prodaj upošteva le štiri attribute vozila in ročno določeno padajočo uteženo funkcijo. Ko bo obseg lastnih, normaliziranih podatkov zadosten, bi bilo treba sedanji, razložljivi pristop dopolniti ali nadomestiti z modeli umetne inteligence (na primer regresijskimi modeli, naključnimi gozdovi ali nevronskimi mrežami), ki znajo zajeti tudi zahtevne, nelinearne odvisnosti med lastnostmi vozila.
- **Ustrezen vir podatkov za komercialno uporabo.** Sedanji zajem javno dostopnih oglasov s spletnih portalov je primeren za raziskovalno uporabo, ni pa trdna ali pravno vzdržna podlaga za komercialno storitev. Za pravo aplikacijo bi bilo treba podatke pridobivati na drugačen način – z dogovorom o souporabi podatkov s portali, prek uradnih vmesnikov ali iz lastnih oglasov, ko bo teh dovolj.
- **Postopna migracija zalednega sistema na TypeScript.** Uvedba statičnega preverjanja tipov bi dolgoročno izboljšala zanesljivost in vzdržljivost zalednega dela aplikacije [22].

- **Nadgradnja infrastrukture in spremljanja delovanja.** Smiselna nadaljnja koraka sta uvedba centraliziranega beleženja dogodkov in spremljanja delovanja sistema (angl. *monitoring*) v produkcijskem okolju. To bi olajšalo odkrivanje napak in analizo obremenitve sistema, ter razširitev nabora avtomatiziranih testov na del aplikacije za vrednotenje vozil in odjemalski del, ki v tem trenutku nista pokrita.
- **Od prototipa do produkcijske aplikacije.** Rezultat tega dela je delujoč prototip, ne pa izdelek, pripravljen za komercialno uporabo. Za to bi bilo treba odpraviti ozko grlo, ugotovljeno pri obremenitvenem testu (ena sama deljena povezava do baze in en strežniški proces; poglavje 5.8) z naborom povezav in več delovnimi procesi, dopolniti pravne vidike (pogoji uporabe, varstvo osebnih podatkov) in razširiti spremljanje delovanja ter testno pokritost.

Literatura

Članki v revijah

- [12] Irfan Darmawan in sod. „Evaluating Web Scraping Performance Using XPath, CSS Selector, Regular Expression, and HTML DOM With Multiprocessing Technical Applications“. V: *JOIV: International Journal on Informatics Visualization* (2022). DOI: 10.30630/joiv.6.4.1525.
- [26] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt in sod. „Array programming with NumPy“. V: *Nature* 585 (2020), str. 357–362. DOI: 10.1038/s41586-020-2649-2.
- [30] John D. Hunter. „Matplotlib: A 2D graphics environment“. V: *Computing in Science & Engineering* 9.3 (2007), str. 90–95. DOI: 10.1109/MCSE.2007.55.
- [33] Marijn Janssen, Yannis Charalabidis in Anneke Zuiderwijk. „Benefits, Adoption Barriers and Myths of Open Data and Open Government“. V: *Information Systems Management* 29.4 (2012), str. 258–268. DOI: 10.1080/10580530.2012.716740.
- [38] Alberto H. F. Laender in sod. „A Brief Survey of Web Data Extraction Tools“. V: *ACM SIGMOD Record* 31.2 (2002), str. 84–93. DOI: 10.1145/565117.565137.
- [45] Gonzalo Navarro. „A Guided Tour to Approximate String Matching“. V: *ACM Computing Surveys* 33.1 (2001), str. 31–88. DOI: 10.1145/375360.375365.

- [46] Christopher Olston in Marc Najork. „Web Crawling“. V: *Foundations and Trends in Information Retrieval* 4.3 (2010), str. 175–246. DOI: 10.1561/15000000017.
- [50] David L. Parnas. „On the Criteria To Be Used in Decomposing Systems into Modules“. V: *Communications of the ACM* 15.12 (1972), str. 1053–1058. DOI: 10.1145/361598.361623. URL: <https://doi.org/10.1145/361598.361623>.

Članki v zbornikih konferenc

- [22] Zheng Gao, Christian Bird in Earl T. Barr. „To Type or Not to Type: Quantifying Detectable Bugs in JavaScript“. V: *Proceedings of the 39th International Conference on Software Engineering (ICSE)*. IEEE Press, 2017, str. 758–769. DOI: 10.1109/ICSE.2017.75.
- [24] Enis Gegic in sod. „Car Price Prediction using Machine Learning Techniques“. V: *TEM Journal*. Zv. 8. 1. 2019, str. 113–118. DOI: 10.18421/TEM81-16.
- [52] Niels Provos in David Mazières. „A Future-Adaptable Password Scheme“. V: *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*. 1999, str. 81–91.

Ostala literatura

- [1] Auto Trader Group. *Auto Trader – New and used cars for sale*. Dostopano: 2. 9. 2026. 2026. URL: <https://www.autotrader.co.uk>.
- [2] AutoScout24. *AutoScout24 – Nova in rabljena vozila*. Dostopano: 22. 8. 2026. 2026. URL: <https://www.autoscout24.com>.
- [3] AvtoNet. *AvtoNet – Oglasnik rabljenih in novih vozil*. Dostopano: 22. 8. 2026. 2026. URL: <https://www.avto.net>.
- [4] Axios Contributors. *Axios – Promise based HTTP client for the browser and node.js*. <https://axios.rest>. Uradna dokumentacija knjižnice. 2024.
- [5] Michael Bayer. *Alembic Documentation*. Dostopano: 2026-07-26. 2024. URL: <https://alembic.sqlalchemy.org/>.
- [6] Stefan Behnel, Martijn Faassen in Ian Bicking. *lxml – XML and HTML with Python*. <https://lxml.de>. Uradna dokumentacija. 2024.
- [7] CARFAX. *CARFAX – Vehicle History Reports and Car Values*. Dostopano: 22. 8. 2026. 2026. URL: <https://www.carfax.com>.
- [8] Cars.com. *Cars.com – New and used cars for sale, prices and values*. Dostopano: 2. 9. 2026. 2026. URL: <https://www.cars.com>.
- [9] BullMQ Contributors. *BullMQ Documentation*. Dostopano: junij 2024. 2024. URL: <https://docs.bullmq.io/>.
- [10] Express.js Contributors. *Express.js Documentation*. Dostopano: junij 2024. 2024. URL: <https://expressjs.com/>.
- [11] Sequelize Contributors. *Sequelize Documentation*. Dostopano: junij 2024. 2024. URL: <https://sequelize.org/>.
- [13] Edsger W. Dijkstra. „On the Role of Scientific Thought“. V: *Selected Writings on Computing: A Personal Perspective*. New York, NY: Springer-Verlag, 1982, str. 60–66. DOI: 10.1007/978-1-4612-5695-3_12.

-
- [14] Docker, Inc. *Docker – Accelerated, Containerized Application Development*. <https://www.docker.com>. Uradna dokumentacija. 2024.
 - [15] Docker, Inc. *Docker Compose Overview*. <https://docs.docker.com/compose/>. Uradna dokumentacija. 2024.
 - [16] Dokploy. *Dokploy Documentation*. <https://docs.dokploy.com>. Uradna dokumentacija. 2024.
 - [17] Ramez Elmasri in Shamkant B. Navathe. *Fundamentals of Database Systems*. 7. izd. Pearson, 2015. ISBN: 9780133970777.
 - [18] Eurostat. *Eurostat – European Statistics*. Dostopano: junij 2025. 2024. URL: <https://ec.europa.eu/eurostat>.
 - [19] Eurotax. *Eurotax*. Dostopano: 20. 8. 2026. 2026. URL: <https://www.eurotax.si>.
 - [20] David Flanagan. *JavaScript: The Definitive Guide*. 7. izd. O'Reilly Media, 2020. ISBN: 9781491952023.
 - [21] Martin Fowler. *Continuous Integration*. <https://martinfowler.com/articles/continuousIntegration.html>. Dostopano: 2026-08-01. 2006.
 - [23] Hector Garcia-Molina, Jeffrey D. Ullman in Jennifer Widom. *Database Systems: The Complete Book*. 2. izd. Pearson, 2008, str. 587–625. ISBN: 9780131873254.
 - [25] GitHub, Inc. *GitHub Actions Documentation*. <https://docs.github.com/en/actions>. Uradna dokumentacija. 2024.
 - [27] Trevor Hastie, Robert Tibshirani in Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2. izd. New York, NY: Springer, 2009. ISBN: 9780387848570. DOI: 10.1007/978-0-387-84858-7.
 - [28] Hetzner Online GmbH. *Hetzner Cloud*. <https://www.hetzner.com/cloud>. Uradna spletna stran. 2024.

-
- [29] Jez Humble in David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA: Addison-Wesley, 2010. ISBN: 9780321601919.
 - [31] Appraisal Institute. *The Appraisal of Real Estate*. 15. izd. Chicago, IL: Appraisal Institute, 2020. ISBN: 9781935328780.
 - [32] ISO. *ISO 3779:2009 – Road vehicles – Vehicle identification number (VIN) – Content and structure*. <https://www.iso.org/standard/52200.html>. Mednarodni standard za zgradbo identifikacijske številke vozila (VIN). 2009.
 - [34] Michael B. Jones, John Bradley in Nat Sakimura. *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force (IETF). Maj 2015. DOI: 10.17487/RFC7519. URL: <https://www.rfc-editor.org/rfc/rfc7519>.
 - [35] Kelley Blue Book. *Kelley Blue Book – New and used car price values*. Dostopano: 2. 9. 2026. 2026. URL: <https://www.kbb.com>.
 - [36] La Centrale. *La Centrale – Annonces de voitures d’occasion et cote auto*. Dostopano: 2. 9. 2026. 2026. URL: <https://www.lacentrale.fr>.
 - [37] Ladjs Contributors. *SuperTest – Super-agent driven library for testing HTTP servers*. <https://github.com/ladjs/supertest>. Uradna dokumentacija. 2024.
 - [39] MailHog. *MailHog*. <https://github.com/mailhog/MailHog>. Uradno programsko skladišče. 2024.
 - [40] Meta Open Source. *Jest – Delightful JavaScript Testing*. <https://jestjs.io/>. Uradna dokumentacija. 2024.
 - [41] Meta Platforms, Inc. *React – The library for web and native user interfaces*. <https://react.dev>. Uradna dokumentacija. 2024.
 - [42] Microsoft. *Playwright – Fast and Reliable End-to-End Testing for Modern Web Apps*. <https://playwright.dev>. Uradna dokumentacija. 2024.

- [43] Microsoft. *TypeScript: JavaScript With Syntax For Types*. Dostopano: junij 2025. 2024. URL: <https://www.typescriptlang.org/>.
- [44] mobile.de. *mobile.de – Gebrauchtwagen und Neuwagen kaufen*. Dostopano: 22. 8. 2026. 2026. URL: <https://www.mobile.de>.
- [47] OpenJS Foundation. *Node.js – JavaScript runtime built on Chrome’s V8 engine*. <https://nodejs.org>. Uradna dokumentacija. 2024.
- [48] Orgesleka. *Used Cars Database – eBay Kleinanzeigen*. Dostopano: junij 2025. 2016. URL: <https://www.kaggle.com/datasets/orgesleka/used-cars-database>.
- [49] OTOMOTO. *OTOMOTO – Ogłoszenia motoryzacyjne*. Dostopano: 2. 9. 2026. 2026. URL: <https://www.otomoto.pl>.
- [51] pmndrs. *Zustand – Getting Started*. <https://docs.pmnd.rs/zustand/getting-started/introduction>. Uradna dokumentacija. 2024.
- [53] Python Software Foundation. *Python 3.11 – What’s New In Python 3.11*. <https://docs.python.org/3/whatsnew/3.11.html>. Uradna dokumentacija. 2022.
- [54] Sebastián Ramírez. *FastAPI Documentation*. Dostopano: 2026-07-26. 2024. URL: <https://fastapi.tiangolo.com/>.
- [55] RapidFuzz Contributors. *RapidFuzz Documentation*. <https://rapidfuzz.github.io/RapidFuzz/>. Uradna dokumentacija. 2024.
- [56] React Hook Form Contributors. *React Hook Form – Performant, flexible and extensible forms*. <https://react-hook-form.com/>. Uradna dokumentacija. 2024.
- [57] Redis Ltd. *Redis Documentation*. Dostopano: junij 2024. 2024. URL: <https://redis.io/docs/latest/>.
- [58] Redux Contributors. *Redux – A JS library for predictable and maintainable global state*. <https://redux.js.org>. Uradna dokumentacija. 2024.

- [59] Leonard Richardson. *Beautiful Soup Documentation*. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>. Uradna dokumentacija. 2024.
- [60] Selenium Project. *Selenium WebDriver Documentation*. <https://www.selenium.dev/documentation/webdriver/>. Uradna dokumentacija. 2024.
- [61] Yaron Sheffer, Dick Hardt in Michael B. Jones. *JSON Web Token Best Current Practices*. Dostopano: junij 2024. 2020. DOI: 10.17487/RFC8725. URL: <https://www.rfc-editor.org/rfc/rfc8725>.
- [62] Statistični urad Republike Slovenije. *Statistični urad Republike Slovenije – SURS*. Dostopano: junij 2025. 2024. URL: <https://www.stat.si/StatWeb/>.
- [63] TanStack. *TanStack Query – Powerful asynchronous state management*. <https://tanstack.com/query/latest>. Uradna dokumentacija. 2024.
- [64] The Linux man-pages project. *crontab(5) — Linux manual page*. Dostopano: 2026-07-26. 2023. URL: <https://man7.org/linux/man-pages/man5/crontab.5.html>.
- [65] The PostgreSQL Global Development Group. *PostgreSQL Documentation: What Is PostgreSQL?* Dostopano: 21. 6. 2026. 2025. URL: <https://www.postgresql.org/docs/current/intro-what-is.html>.
- [66] Vite Contributors. *Vite – Next Generation Frontend Tooling*. <https://vite.dev>. Uradna dokumentacija orodja. 2024.
- [67] Evan Wallace. *esbuild – An extremely fast bundler for the web*. <https://esbuild.github.io>. Uradna dokumentacija orodja. 2024.
- [68] Webpack Contributors. *webpack – Module Bundler*. <https://webpack.js.org>. Uradna dokumentacija. 2024.
- [69] Adam Wiggins. *The Twelve-Factor App*. Dostopano: 2026-08-01. 2017. URL: <https://12factor.net/>.

Dodatek

Koda: Definicija modela User

Izsek kode 8: Definicija modela User z ORM knjižnico Sequelize.

```
1 import { DataTypes } from "sequelize";
2 import sequelize from "../config/db.js";
3
4 const User = sequelize.define(
5   "User",
6   {
7     id: {
8       type: DataTypes.INTEGER,
9       autoIncrement: true,
10      primaryKey: true,
11    },
12    firstName: {
13      type: DataTypes.STRING,
14      allowNull: false,
15    },
16    lastName: {
17      type: DataTypes.STRING,
18    },
19    email: {
20      type: DataTypes.STRING,
21      allowNull: false,
22      unique: true,
23      validate: {
24        isEmail: true,
25      },
26    },
27    telephone: {
28      type: DataTypes.STRING,
```

```
29     allowNull: false,
30   },
31   password: {
32     type: DataTypes.STRING,
33     allowNull: false,
34   },
35   soldCars: {
36     type: DataTypes.INTEGER,
37     defaultValue: 0,
38   },
39 },
40 { timestamps: true },
41 );
42
43 export default User;
```

Izsek kode 9: Primer normalizacije podatkov

```
1 def extract_data_from_raw_payload(raw
2   valid = True
3   data = {}
4
5   raw_brand = raw_payload.get("brand", None)
6   raw_model = raw_payload.get("model", None)
7
8   if not raw_brand:
9     valid = False
10  if not raw_model:
11    valid = False
12
13  if valid:
14    brand_match = match_brand(_preprocess(raw_brand), SOURCE)
15    if brand_match is None:
16      print(f"Could not match brand '{raw_brand}' for raw_id:
17        ↳ {id}")
18      valid = False
19    else:
20      brand_id, brand_canonical = brand_match
21      data["brand_name"] = brand
22
23      model_match = match_model
24      _preprocess(raw_model), brand_id, brand_canonical, SOURCE
```

```
24         )
25         if model_match is None:
26             print(
27                 f"Could not match model '{raw_model}' (brand:
                ↪ '{brand_canonical}') for raw_id: {id}"
28             )
29             valid = False
30         else:
31             _, model_canonical = model_match
32             data["model_name"] =
```